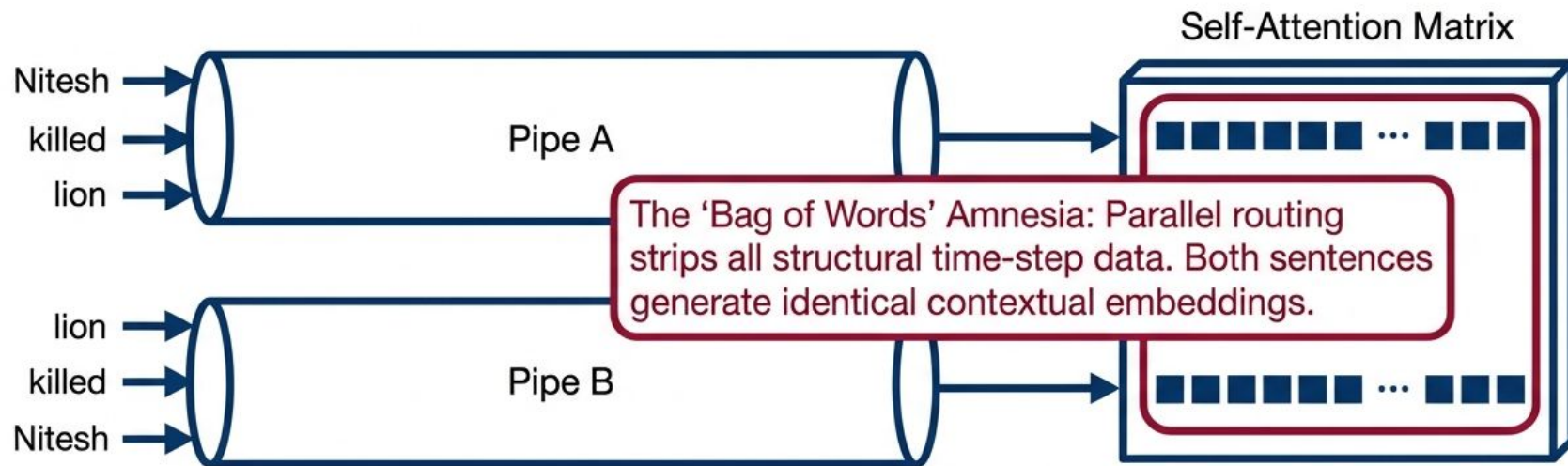


Join the lecture online on your dashboard.

Recap : Positional Encoding

The Blind Spot of Parallel Processing



To make real-world NLP applications viable, we must artificially inject sequential order back into the embedding space before self-attention occurs.

Q : If you were handed a stack of 'n' index cards, and each card had one word from a sentence written on it, what is the most straightforward way you could mark those cards so someone else could reconstruct the exact order of the sentence?

A : Just mark each card with a number from 1 to n

Insight : In a Transformer, we do something similar. Since the model sees all words at once, we need to "stamp" each word vector with information about its location. However, we can't just add a simple integer like 1 or 100 directly to the data.

If we simply added 1, 2, 3, ... to the word embeddings, the numbers could get very large for long sentences. This might "overwhelm" the actual meaning of the word or make it hard for the model to handle sequences longer than what it saw during training

Instead, Transformers use **Positional Encoding** (PE), which uses sine and cosine functions of different frequencies to create a unique "signature" for every position.

The formula looks like this for a position **pos** and dimension **i**:

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

The Mathematical Requirements for Positional Identity

1. Bounded & Stable

Must output values strictly within a normalized range (e.g., -1 to 1) to survive Backpropagation without causing exploding gradients.

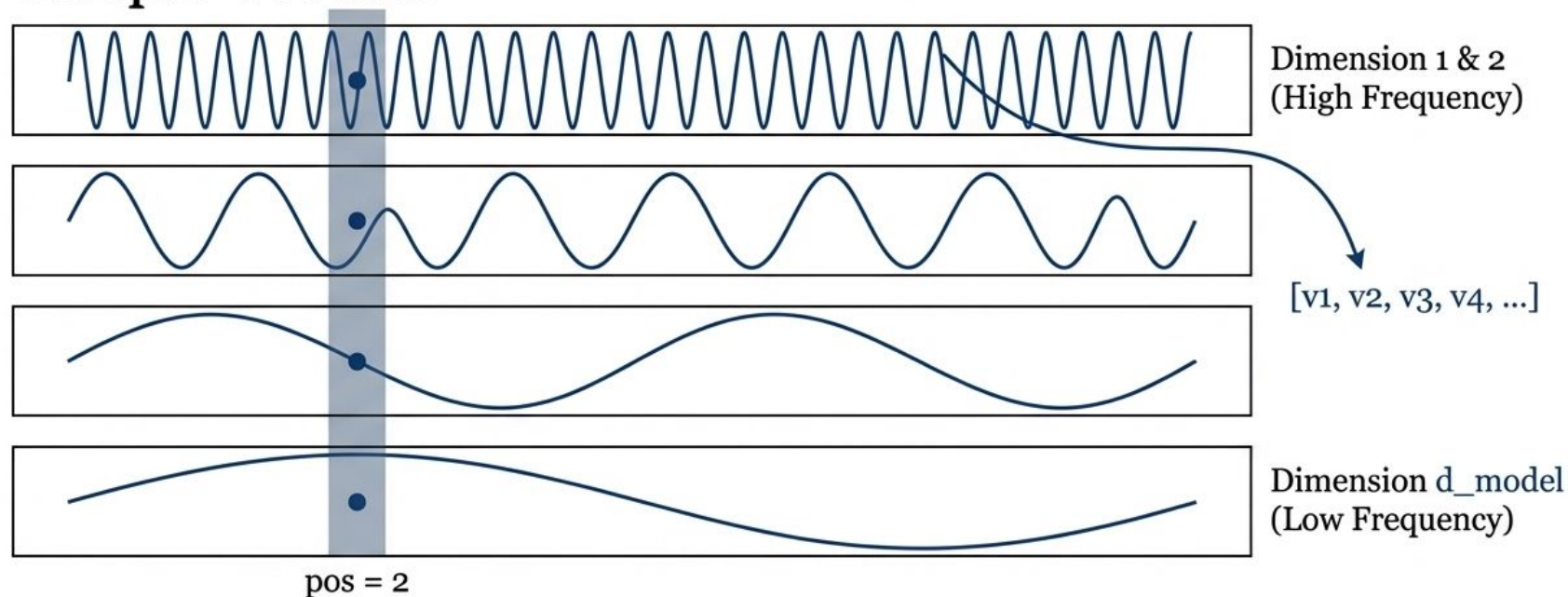
2. Continuous & Smooth

Must avoid discrete jumps (1, 2, 3) which disrupt the gradient flow. Neural networks optimize best over continuous, differentiable functions.

3. Absolute & Consistent

A specific position (e.g., pos = 4) must output the exact same unique mathematical signature regardless of whether the document is 10 words or 10,000 words long.

Attempt 5: Decreasing Frequencies for Infinite Unique Vectors



By stacking multiple Sine/Cosine pairs and progressively decreasing their frequencies, we create a deeply multi-dimensional vector. High frequencies capture precise local differences, while low frequencies capture macro sequence placement.

Q : Once we have this "signature" for a word's position, how do you think we combine it with the actual word data (the embedding) so the model sees both the meaning and the position at the same time?

A : We combine them through vector addition.

Instead of adding the position info as a separate column or a single number, we add the positional encoding vector directly to the word embedding vector.

If the word "apple" is represented by a vector v_apple and its position in the sentence is $pos=3$,

we calculate: $v_final = v_apple + PE_3$

$$PE_{(pos, 2i)} = \sin \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

$$PE_{(pos, 2i+1)} = \cos \left(\frac{pos}{10000^{2i/d_{model}}} \right)$$

Architectural Integration: Addition vs. Concatenation

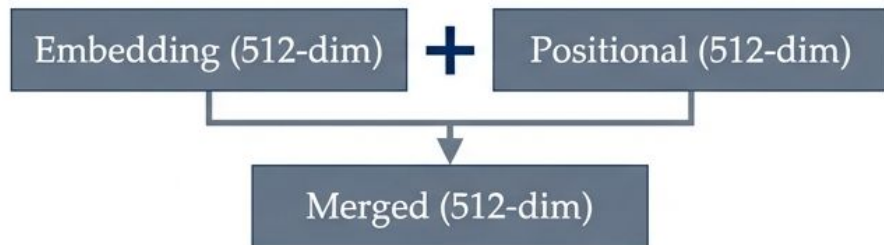
Concatenation (The Intuitive Method)



Dimensions = $2 \times d_{model}$
Parameters = Doubled
Training Time = $2 \times$



Element-wise Addition (The "Attention Is All You Need" Method)



Dimensions = d_{model}
Parameters = Unchanged
Training Time = Unchanged



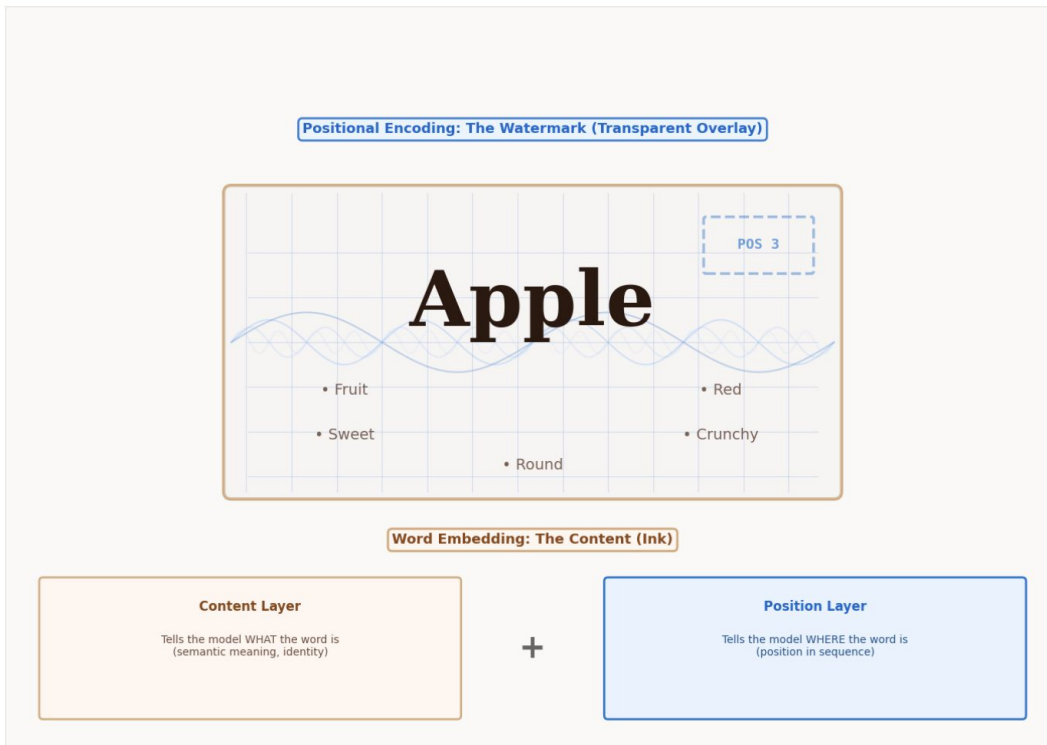
The Golden Rule: Because we use vector addition, the Positional Encoding dimension must exactly equal the Word Embedding dimension ($d_{model} = 512$).

Why addition works

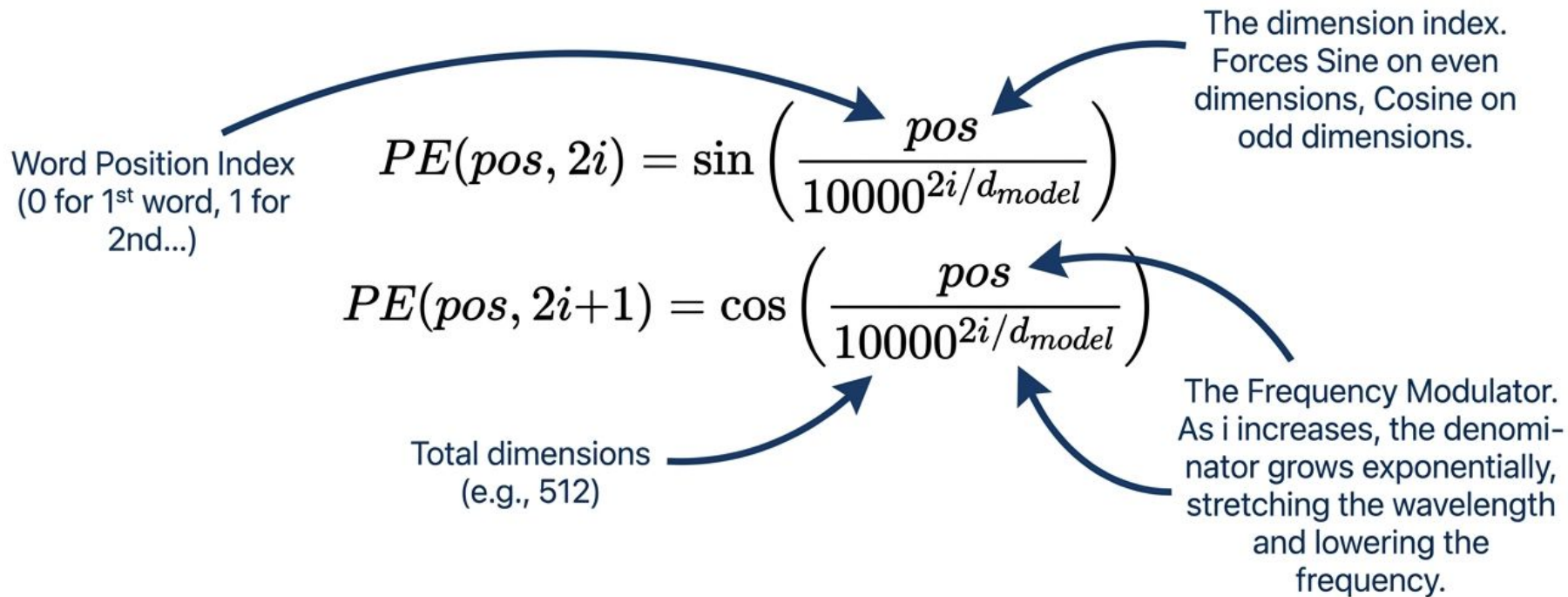
It might seem like adding these numbers together would "smudge" or corrupt the original word meaning. However, because these vectors exist in high-dimensional space (often 512 dimensions), the model can learn to treat the "meaning" and the "position" as separate features.

Imagine it like a transparent overlay:

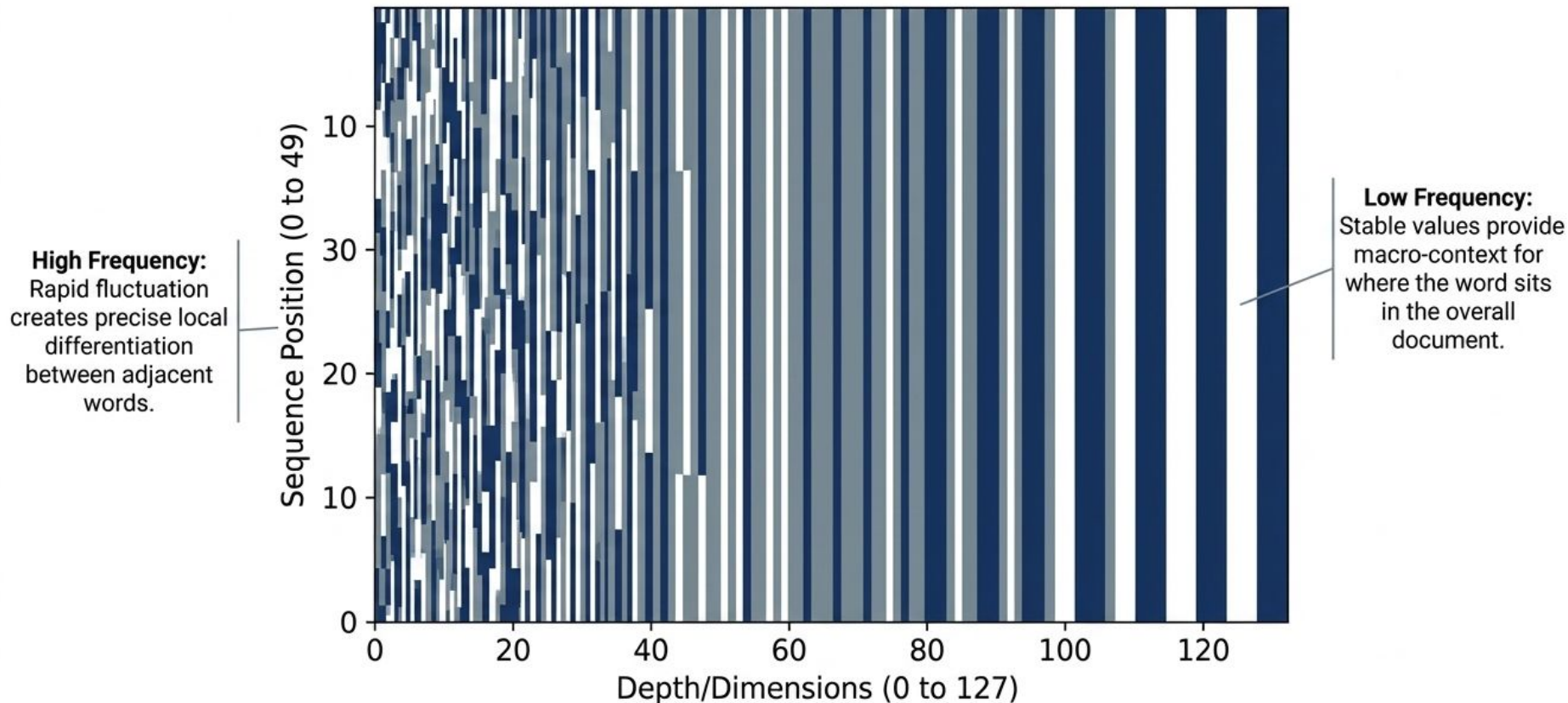
- The word embedding tells the model the "identity" of the word (it's a fruit, it's red, it's crunchy).
- The positional encoding acts like a faint watermark or grid coordinate over the word.



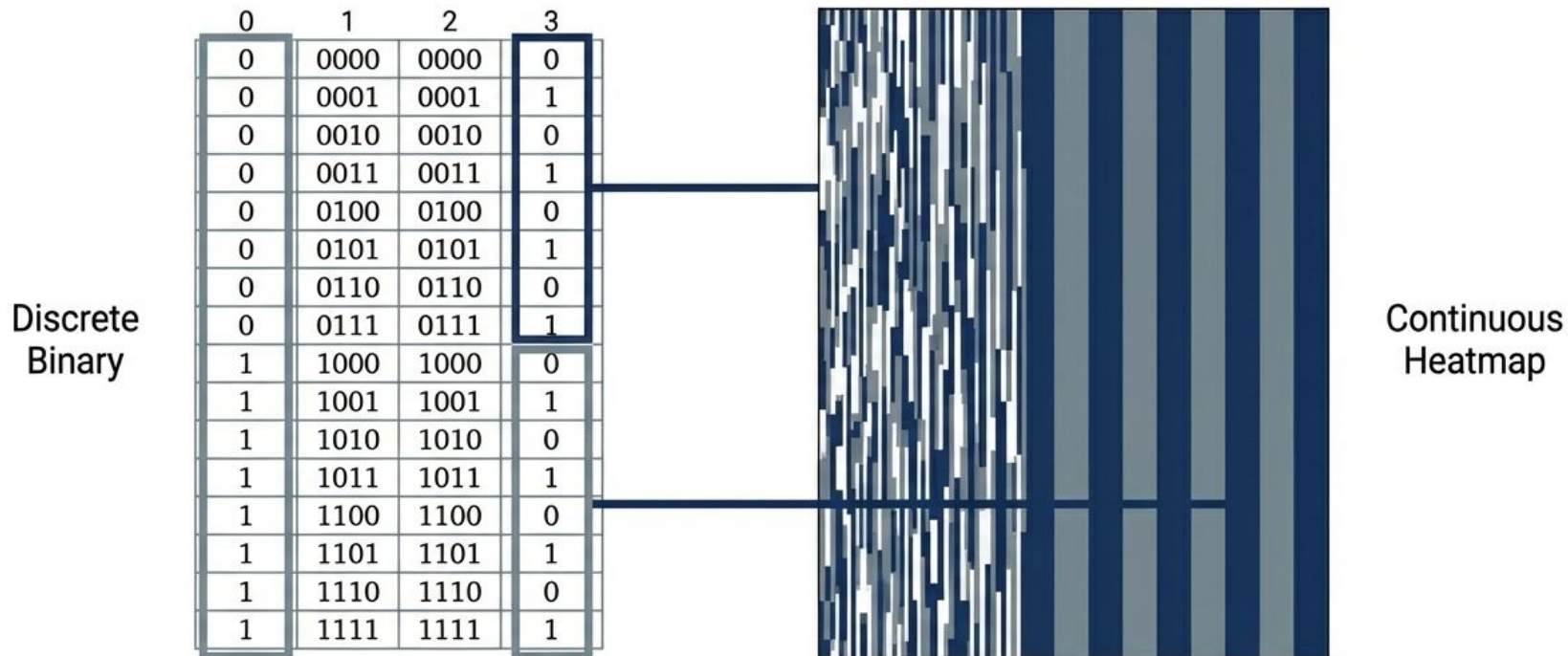
Deconstructing the “Attention is All You Need” Formula



Visualizing the Positional Encoding Matrix

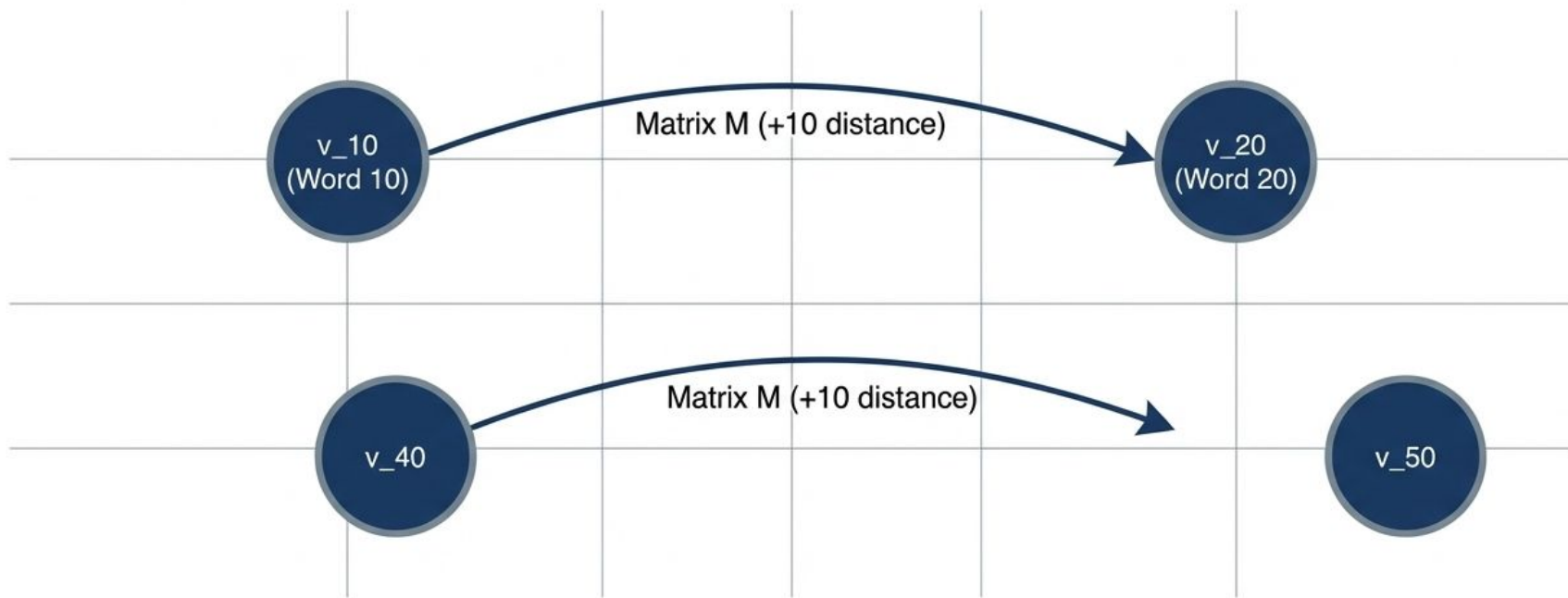


Conceptual Analogy: Continuous Binary Encoding



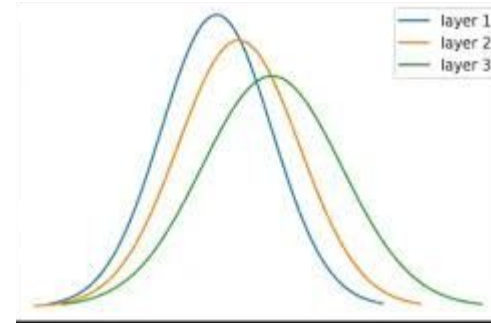
The brilliant intuition behind the math: it is simply binary counting translated into the domain of continuous floating-point numbers. It replicates the bit-flipping pattern without breaking neural network continuity requirements.

The Hidden Superpower: Relative Position Awareness



While our system assigns absolute positions (1, 2, 3), language requires relative understanding (e.g., “**how far is lion from killed?**”). Because of the trigonometric properties of **Sine** and **Cosine**, the network can easily learn to calculate the distance between any two words using a single linear transformation matrix.

In deep networks, as data passes through many layers, the values in the vectors can sometimes become **very large or very small** (a problem often called **internal covariate shift**). This makes training unstable and slow.

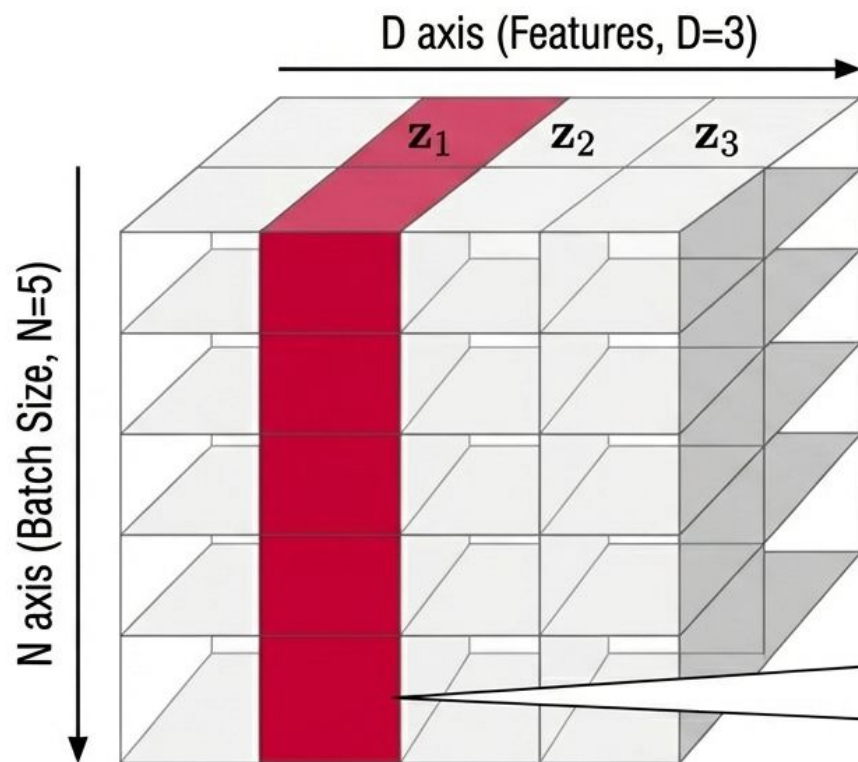


Q : How do we fix this ?

Layer Normalization

Baseline: The Mechanics of Batch Normalization

Computing statistics across the N dimension (Batch Size).



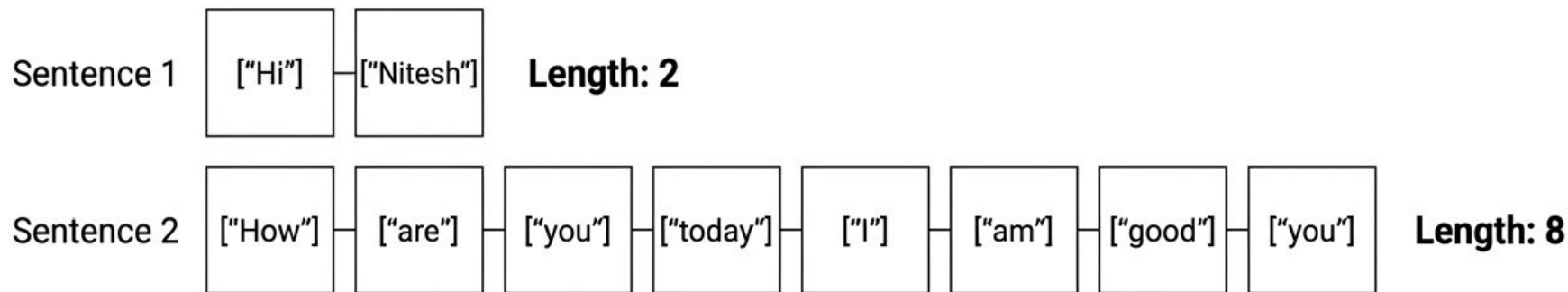
For a mini-batch B of size m , we calculate the empirical mean and variance strictly per feature/node:

$$\mu_B^{(j)} = \frac{1}{m} \sum_{i=1}^m z_i^{(j)}$$

$$(\sigma_B^{(j)})^2 = \frac{1}{m} \sum_{i=1}^m (z_i^{(j)} - \mu_B^{(j)})^2$$

Batch Normalization collapses the batch dimension to find the mean and variance for a single feature dimension across all disparate samples.

The Batching Challenge: Variable Sequence Lengths



GPU architecture dictates that data must be processed in uniform, contiguous blocks (tensors).

To process a batch of text simultaneously, the data structure must form a perfect matrix.

The Engineering Bottleneck

In NLP, batching naturally occurring text results in ragged arrays. A single batch might contain a 2-word sentence and a 100-word sentence.

Standard linear algebra operations cannot compute over non-uniform tensor dimensions.

Statistical Distortion and Covariate Noise

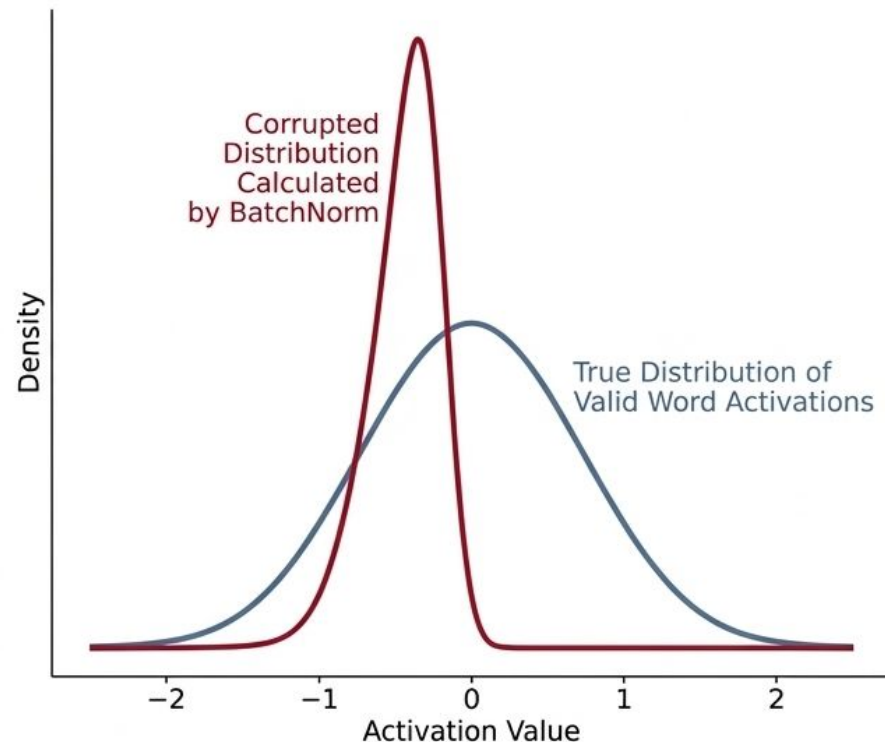
By forcing padding zeros into the batch-wise mean and variance calculations, BatchNorm fundamentally violates the core purpose of normalization.

1. Corrupted Representation:

The computed mean is artificially dragged toward zero, completely misrepresenting the true activation distribution of actual language tokens.

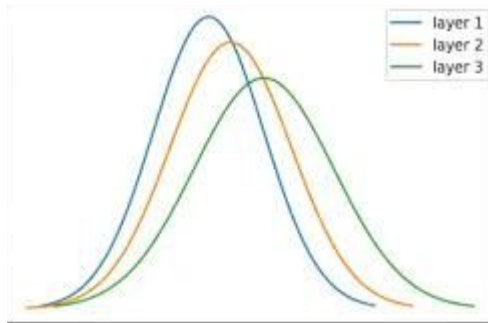
2. Batch Dependency:

The statistics of Sentence 1 ("Hi Nitesh") are now highly dependent on the length of Sentence 2. If Sentence 2 changes, the padding requirement changes, entirely altering the μ and σ applied to Sentence 1.



Conclusion: Batch Normalization treats engineering artifacts (padding) as mathematical signals, rendering it incompatible with sequential NLP data.

In deep networks, as data passes through many layers, the values in the vectors can sometimes become **very large or very small** (a problem often called **internal covariate shift**). This makes training unstable and slow.



Q : How do we fix this ?

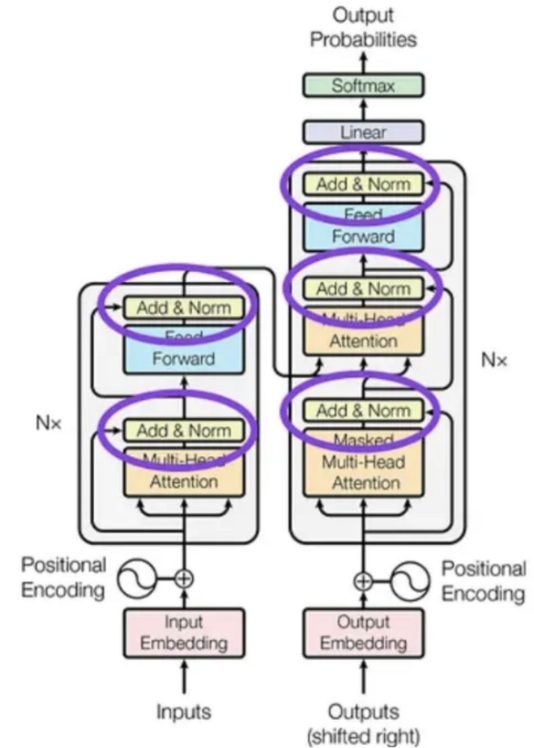
Layer Normalization fixes this by "re-centering" the values. For every word vector, it:

1. Calculates the **mean** and **variance** of all the numbers in that specific vector.
2. Normalizes the values so they have a mean of 0 and a variance of 1.
3. Applies two learnable parameters, **gamma** and **beta** to scale and shift the data back to a range that the model finds most useful for learning.

Q : If we didn't use normalization and just let the raw numbers from our addition ($v_apple + PE_3$) flow through dozens of layers of matrix multiplications, what do you think might happen to the **size** of the numbers in those vectors?

Ans : Internal covariate shift will occur as value range increases rapidly. Therefore we use Layer Normalization to **stabilize and accelerate training**.

Let's delve deeper into this with a well-known diagram of the transformer architecture. You'll notice that the normalization step is applied right after the attention mechanism.



Layer Normalization Example

For each input x of the layer, Layer Normalization does the following:

- Mean and Variance for Each vector

$$\mu = \frac{1}{H} \sum_{i=1}^H x_i$$
$$\sigma^2 = \frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2$$

- Normalize the Input

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

- Apply Scaling and Shifting

$$y_i = \gamma \hat{x}_i + \beta$$

Embeddings + Positional Encoded Vectors

$$x_1 = [3.0, 5.0, 2.0, 8.0]$$

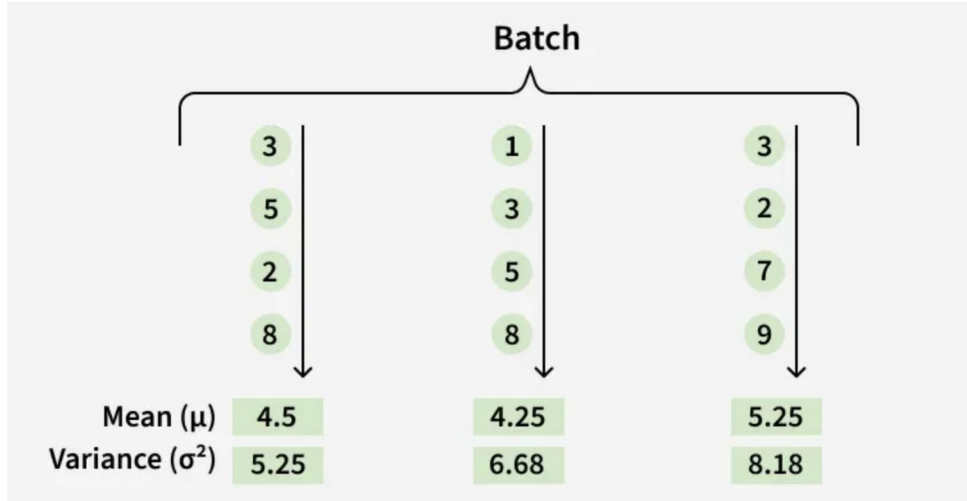
$$x_2 = [1.0, 3.0, 5.0, 8.0]$$

$$x_3 = [3.0, 2.0, 7.0, 9.0]$$

Layer Normalization Example : Mean and Variance

For each input x of the layer, Layer Normalization does the following:

- Mean and Variance for Each vector



Embeddings + Positional Encoded Vectors

$$x1 = [3.0, 5.0, 2.0, 8.0]$$

$$x2 = [1.0, 3.0, 5.0, 8.0]$$

$$x3 = [3.0, 2.0, 7.0, 9.0]$$

Layer Normalization Example

For each input x of the layer, Layer Normalization does the following:

- Normalize the Input

For $x_1 = [3.0, 5.0, 2.0, 8.0]$:

$$x'_1 = \left[\frac{3.0-4.5}{\sqrt{5.25+1e-5}}, \frac{5.0-4.5}{\sqrt{5.25+1e-5}}, \frac{2.0-4.5}{\sqrt{5.25+1e-5}}, \frac{8.0-4.5}{\sqrt{5.25+1e-5}} \right]$$

We calculate each normalized value for x_1 :

$$x'_1 = [-0.6547, 0.2182, -1.0911, 1.5275]$$

Similarly :

$$x'_2 = [-1.2568, -0.4834, 0.2900, 1.4501]$$

$$x'_3 = [-0.7863, -1.1358, 0.6116, 1.3106]$$

Embeddings + Positional Encoded Vectors

$$x_1 = [3.0, 5.0, 2.0, 8.0]$$

$$x_2 = [1.0, 3.0, 5.0, 8.0]$$

$$x_3 = [3.0, 2.0, 7.0, 9.0]$$

Layer Normalization Example

For each input x of the layer, Layer Normalization does the following:

- Apply Scaling and Shifting

To ensure that the normalized activations can still represent a wide range of values, learnable parameters γ (scaling) and β (shifting) are introduced. Final output y is computed as:

$$y_i = \gamma \hat{x}_i + \beta$$

In real Layer Normalization, γ and β are not scalars. They are vectors of size H (number of features). Each feature has its own learnable γ and β .

Here let's assume $\gamma = 1.5$ $\beta=0.5$ for simplicity

Embeddings + Positional Encoded Vectors

$x_1 = [3.0, 5.0, 2.0, 8.0]$

$x_2 = [1.0, 3.0, 5.0, 8.0]$

$x_3 = [3.0, 2.0, 7.0, 9.0]$

Layer Normalization Example

For each input x of the layer, Layer Normalization does the following:

- Apply Scaling and Shifting

To ensure that the normalized activations can still represent a wide range of values, learnable parameters γ (scaling) and β (shifting) are introduced. Final output y is computed as:

$$y_i = \gamma \hat{x}_i + \beta$$

Here let's assume $\gamma = 1.5$ $\beta = 0.5$

- For x_1 : $y_1 = [-0.4820, 0.8273, -1.1366, 2.7913]$
- For x_2 : $y_2 = [-1.3851, -0.2250, 0.9350, 2.6751]$
- For x_3 : $y_3 = [-0.6795, -1.2037, 1.4174, 2.4658]$

Embeddings + Positional Encoded Vectors

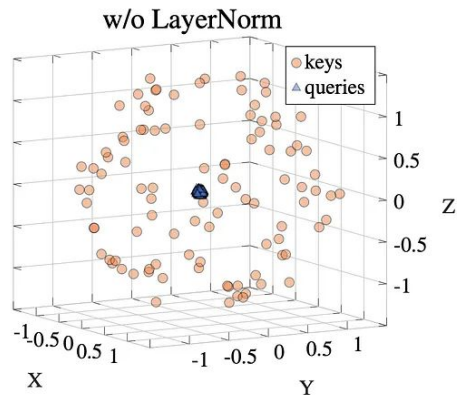
$x_1 = [3.0, 5.0, 2.0, 8.0]$

$x_2 = [1.0, 3.0, 5.0, 8.0]$

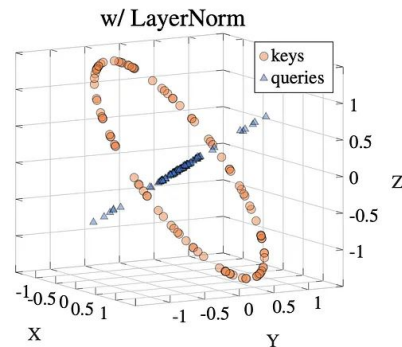
$x_3 = [3.0, 2.0, 7.0, 9.0]$

Advantages of Layer Normalization

1. **Works with Small Batches:** It is not dependent on batch size which make it ideal for cases with small batches or when working with reinforcement learning where each input is processed individually.
2. **Stabilizes Learning:** It normalizes activations within a layer helping to prevent issues like exploding and vanishing gradients which ensures smoother and more efficient training.
3. **Independent of Batch Size:** Unlike Batch Normalization, it is calculated for each individual data point helps in making it more flexible and suitable for situations with variable batch sizes.



(a) Without LayerNorm, the model has learned key and query vectors without any apparent geometric structure.



(b) LayerNorm projects the key vectors onto the same hyperplane so that the model can learn to align the queries to be orthogonal to the keys.

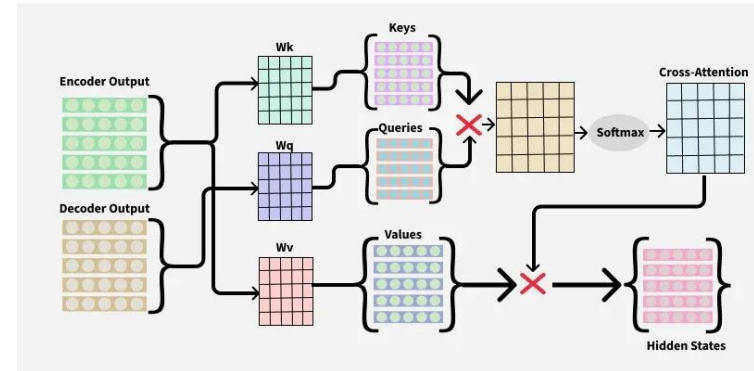
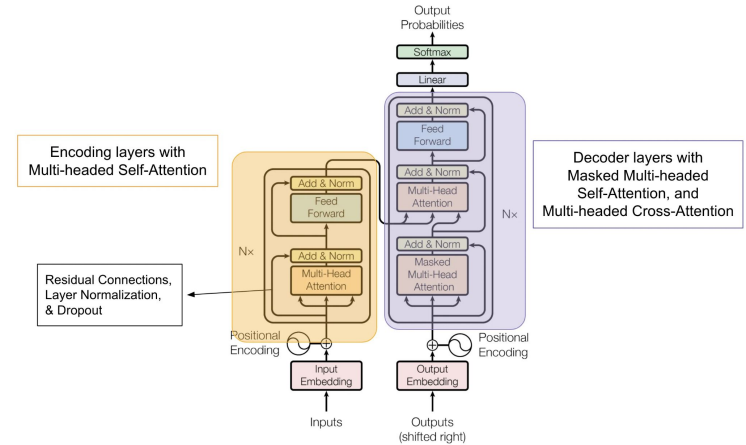
Dimension	Batch Normalization	Layer Normalization
Computation Axis	Vertical (Across Batch N)	Horizontal (Across Features D)
Mean / Var Calculation	$\mu_{\mathcal{B}}^{(j)}, \sigma_{\mathcal{B}}^{(j)}$ (Per Feature)	μ_i, σ_i (Per Token)
Learnable Parameters	$\gamma^{(j)}, \beta^{(j)}$ (Per Feature)	γ, β (Vector over D dimensions)
Sequence Lengths	Highly sensitive to variable lengths	Completely agnostic to sequence length
Padding Tokens	Corrupts global batch statistics	Isolated computation; zero interference
Batch Size Dependency	Fails with very small batch sizes	Operates identically even for batch size $N=1$
Transformer Placement	Generally incompatible	Post-Attention / Post-FFN (Add & Norm)

The transition to Layer Normalization is not merely an architectural preference; it is a mathematical prerequisite for processing unaligned, variable-length sequential data.

Now that our data is "stamped" with its position and "stabilized" by normalization, it enters the heart of the model.

The key parts and concepts of the model :

1. **Encoder Architecture** : Take the input sentence and create a rich numerical representation of what every word means in context.
2. **Masked Self-Attention** : Mechanism that prevents decoder from 'cheating' by masking appropriate future entries
3. **Decoder Architecture** : Takes the information from the Encoder and uses it to generate a response, one word/token at a time.
4. **Cross Attention** : Mechanism that allows encoder and decoder to communicate with each other and develop a unified robust understanding to produce better responses.



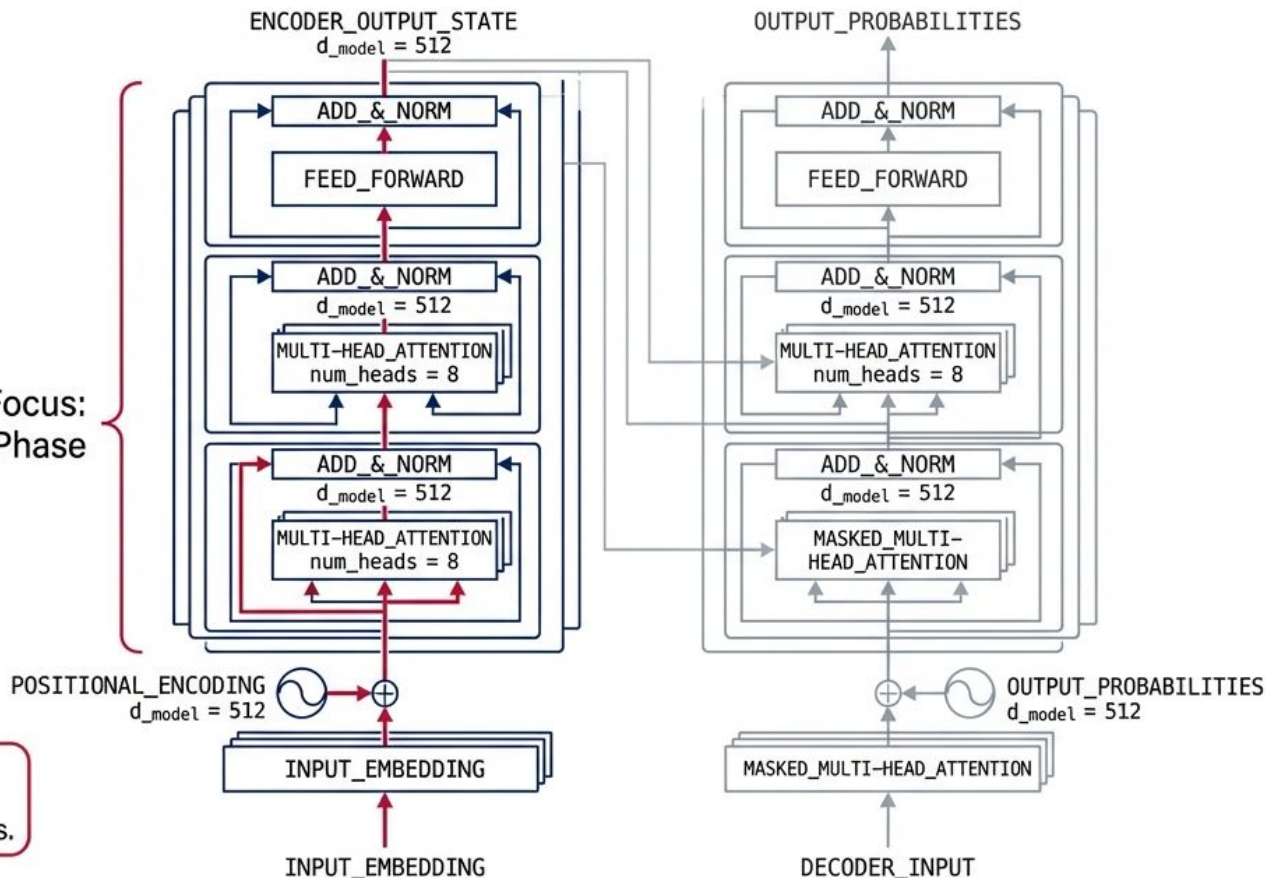
Encoder Architecture

The Transformer Macro-Architecture: Isolating the Encoder Stack

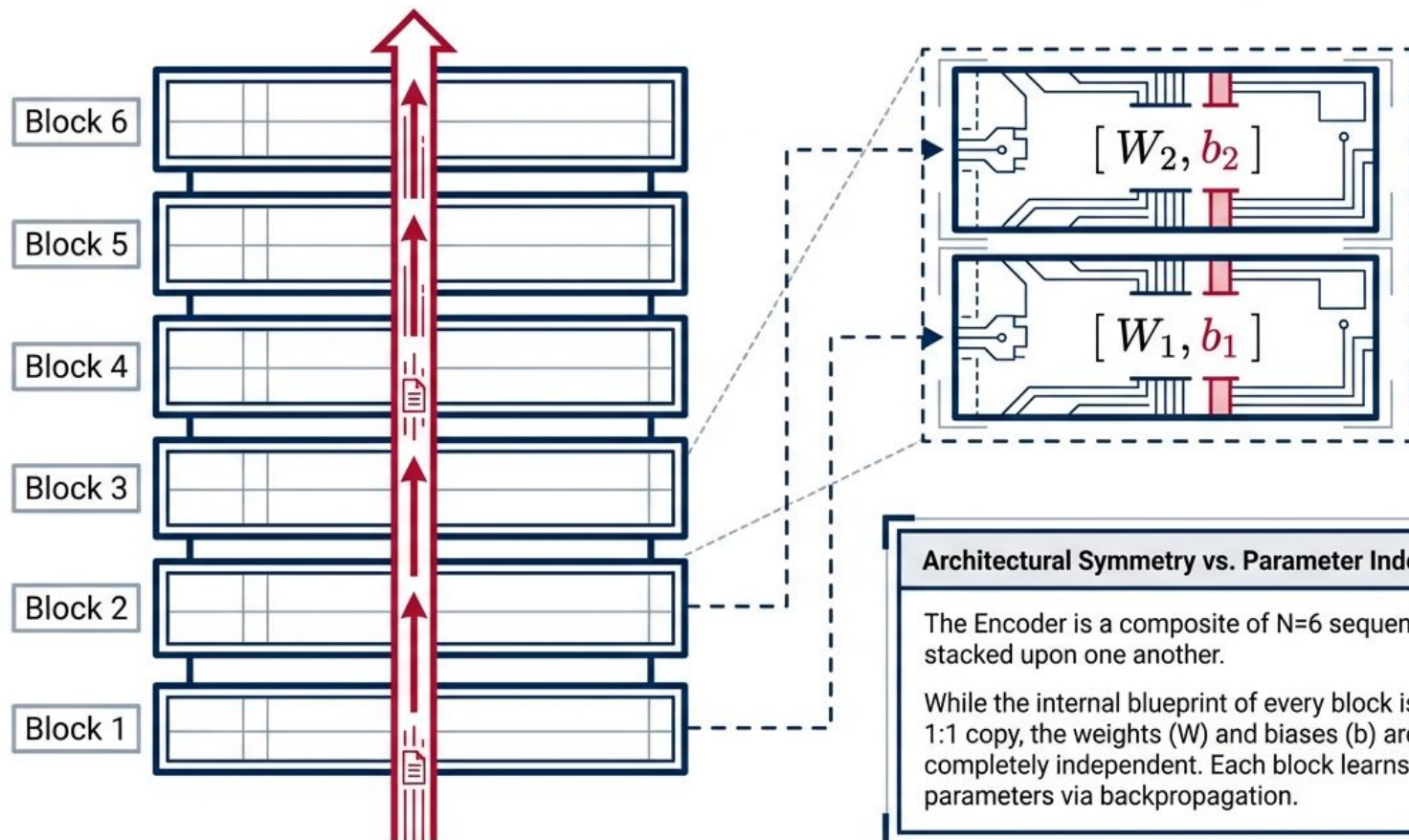
- The Transformer is not a monolith; it is a coupled bipartite system.
- The Encoder (left) maps raw inputs to continuous, context-rich representations. The Decoder (right) generates predictions.

System Focus:
The Encoder Phase

Current State: Focusing exclusively on mapping inputs to continuous state representations before generation begins.



The Encoder Stack Topology (N=6)



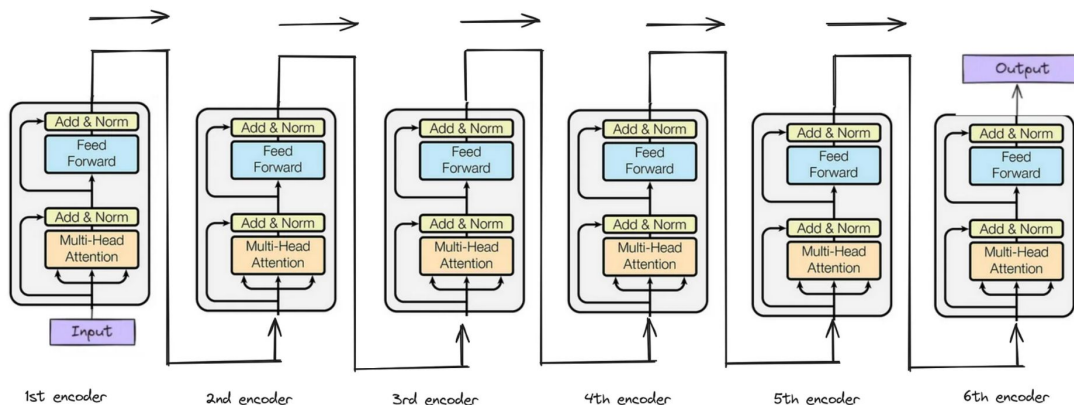
Architectural Symmetry vs. Parameter Independence

The Encoder is a composite of N=6 sequential layers stacked upon one another.

While the internal blueprint of every block is a perfect 1:1 copy, the weights (W) and biases (b) are completely independent. Each block learns unique parameters via backpropagation.

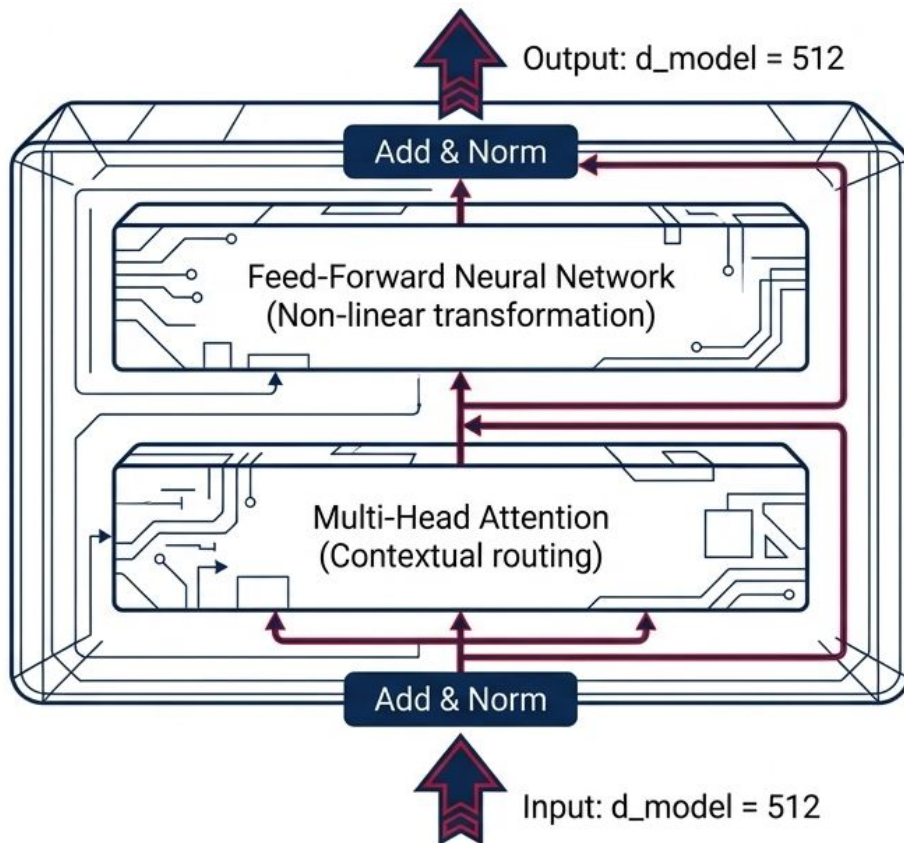
The **Encoder** is the part of the Transformer that acts like a sophisticated reader. Its main goal is to take a sequence of words and turn them into a set of "context-aware" vectors.

The input data, typically a batch of sentences, enters the first encoder block, undergoes processing, and the output moves to the next encoder block.



Flow of data : output of one encoder is input for next encoder block

Micro-Architecture of a Single Encoder Block

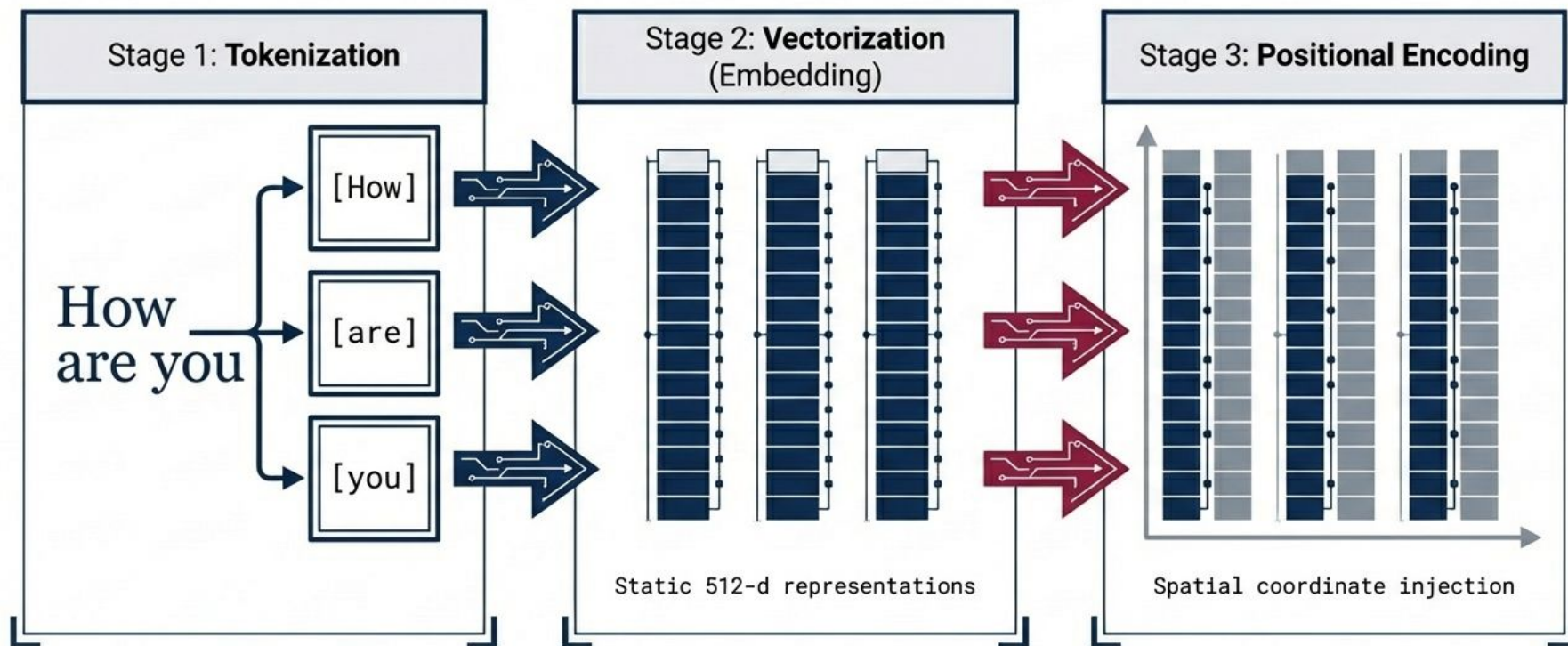


Every block routes data sequentially through two distinct sub-layers.

The tensor enters and exits the block with the exact same dimensions, allowing for infinite vertical stackability.

The Input Pre-Processing Pipeline

Machines cannot compute discrete text. The raw string must be **tokenized**, mapped to continuous space, and injected with positional context.

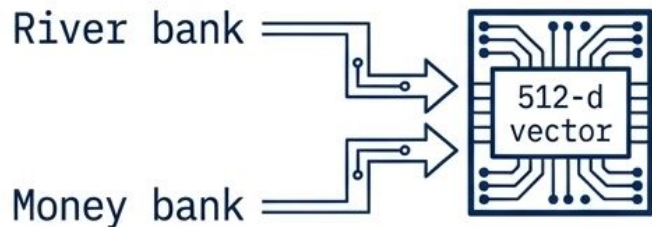


Before diving into the main parts of the encoder, it's crucial to understand the input block, where three essential operations are performed. These steps take place before the input is fed into the encoder:

- **Tokenization:** The first operation is tokenization. For instance, if our sentence is "How are you?", it gets tokenized into "How", "are", and "you".
- **Text Vectorization (Embedding):** We use word embeddings to map each word to a vector. In our case, every word is represented as a 512-dimensional vector. For example, "How" becomes a vector of 512 numbers, "are" gets its own vector, and "you" gets another.
- **Positional Encoding:** The position of each word in the sentence impacts its meaning. This is where **positional encoding** comes in.

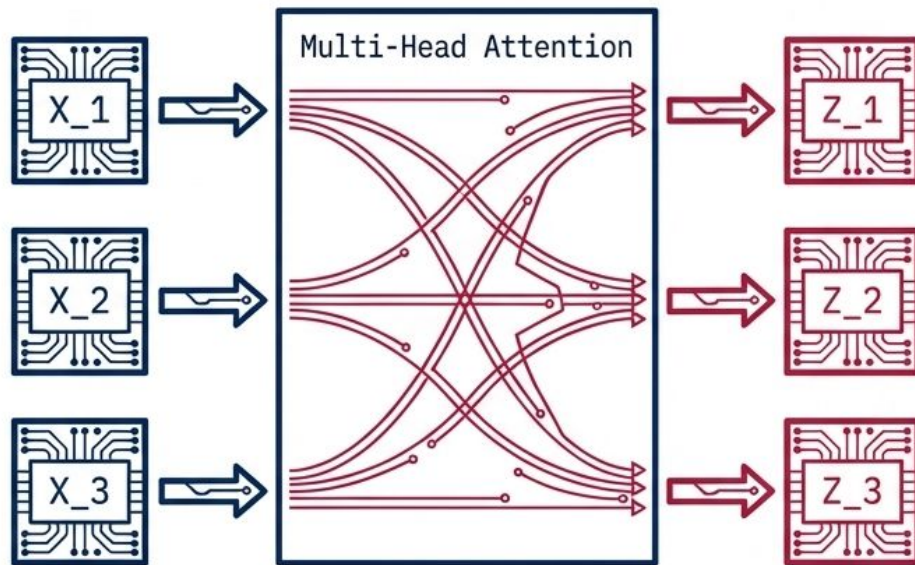
Sub-Layer 1: Multi-Head Attention (MHA)

The Static Embedding Problem



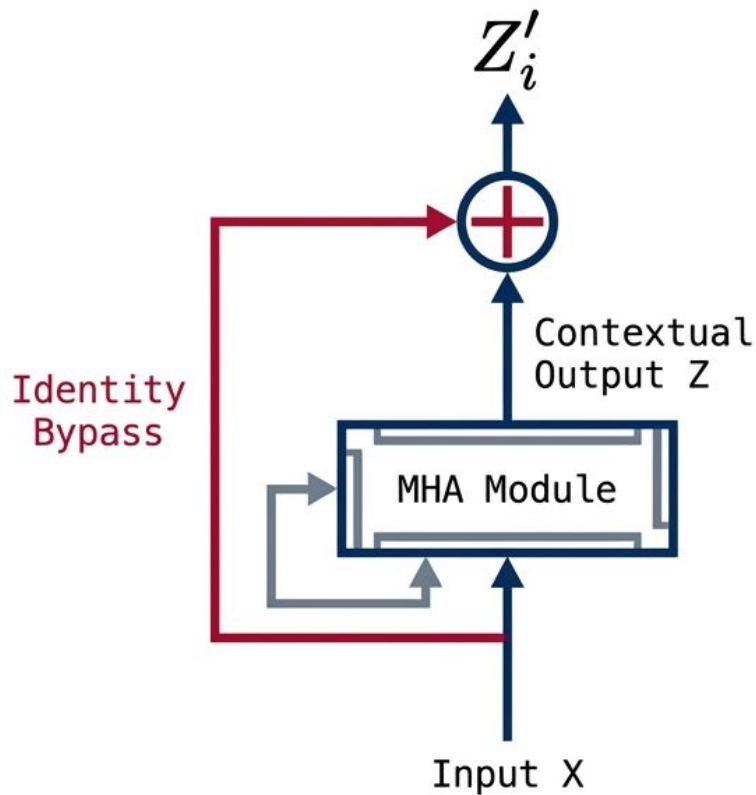
Identical words map to identical vectors regardless of context.

Contextual Intertwining (MHA)



Each word attends to surrounding words, outputting a contextually aware 512-dimensional representation.

Sub-Layer 1 Post-Processing: The Residual Connection

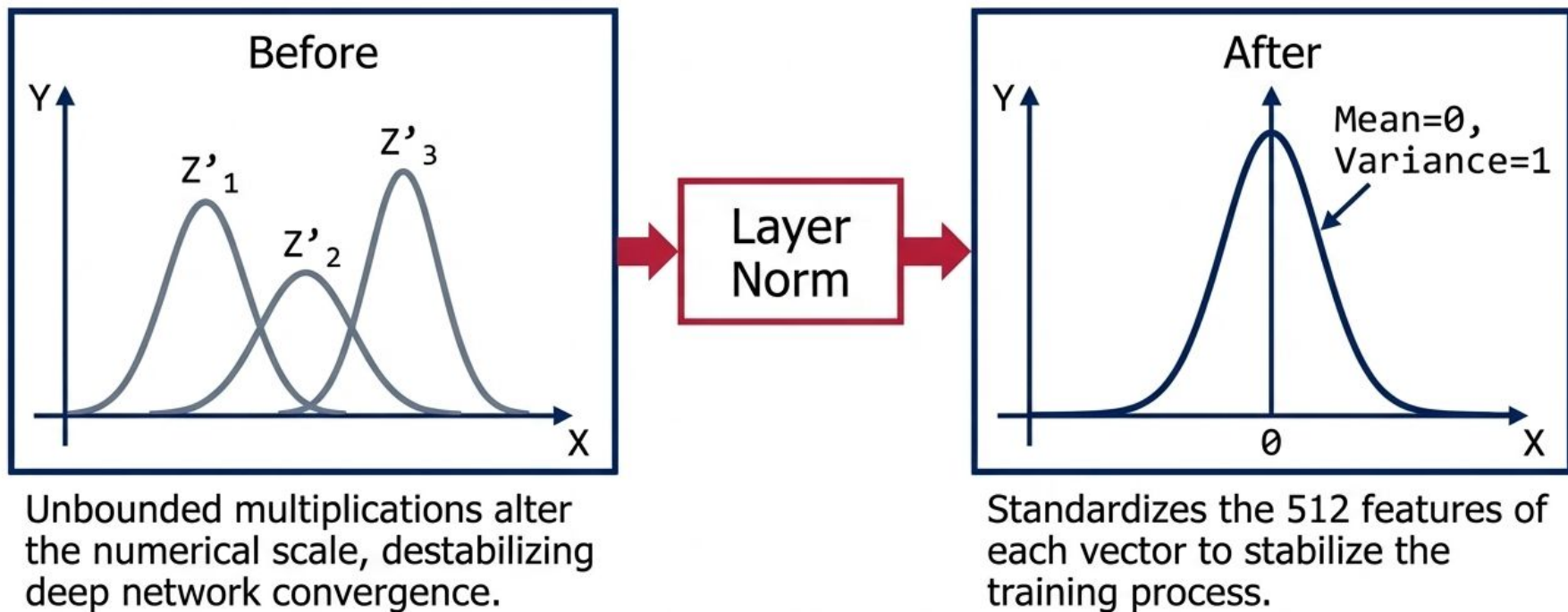


$$Z'_i = Z_i + X_i$$

Function: A skip connection routes the original input around the sub-layer and adds it directly to the contextualized output.

Purpose: Preserves original feature identity and provides an unhindered alternate path for gradient flow during backpropagation.

Sub-Layer 1 Post-Processing: Layer Normalization



$$[3 \times 512] \rightarrow \text{Layer Norm} \rightarrow Z_{\text{norm}} [3 \times 512]$$

Once we have the positional encodings combined with the input embeddings, the next step is to pass these vectors through the first encoder block.

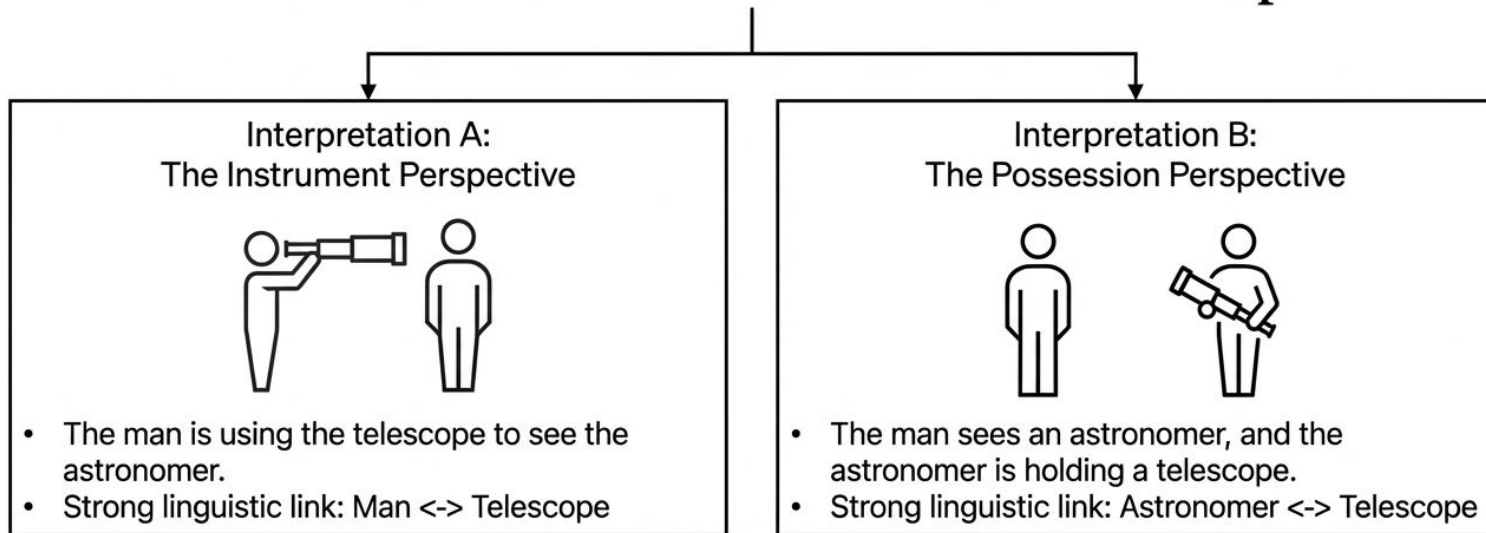
We'll focus on two key operations happening in the encoder: Multi-head Attention and Normalization.

1. **Multi-head Attention** : Instead of relying on just one self-attention mechanism, multi-head attention runs multiple self-attention operations in parallel. This allows the model to focus on different aspects of the sentence simultaneously, creating a more diverse and context-aware representation of the input.
2. **Layer Normalization** : Layer Normalization fixes ICS by "re-centering" the values.

Multi-Head Attention Recap

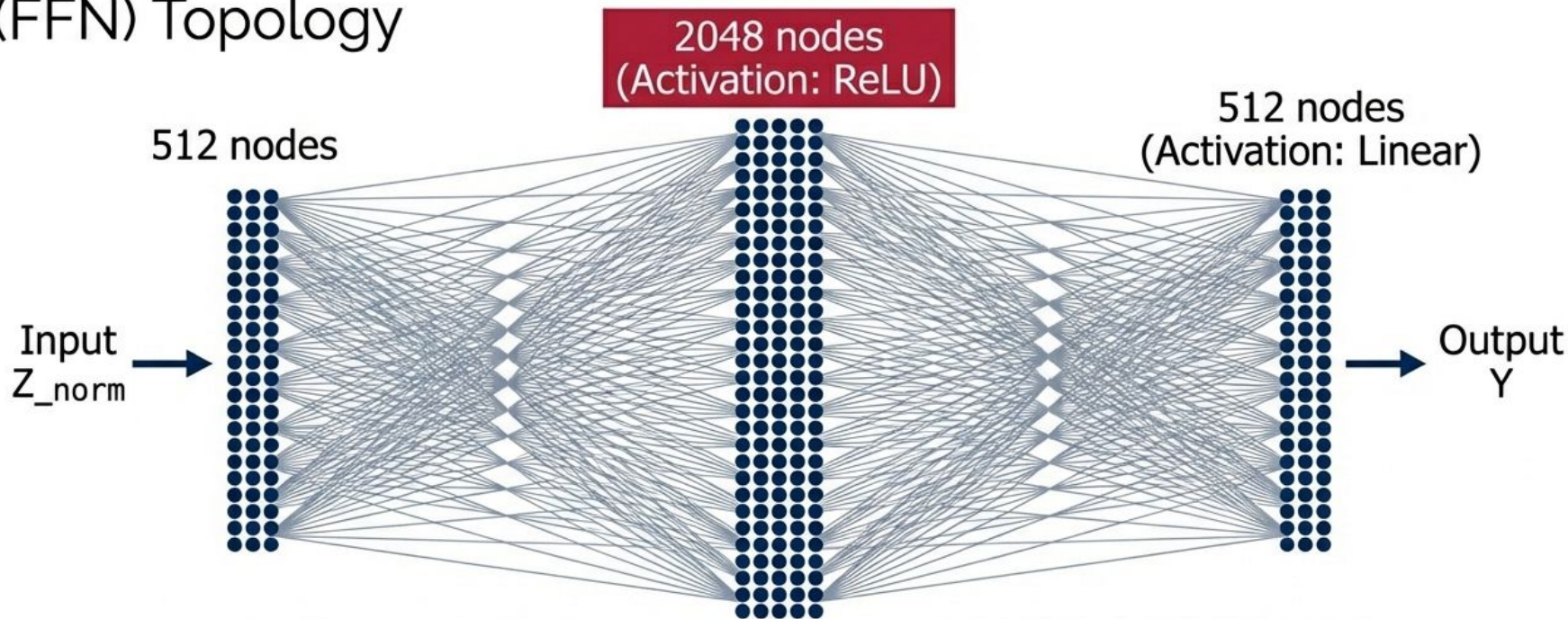
The Limitation: Semantic Ambiguity

The man saw the astronomer with a telescope.



Natural Language is inherently ambiguous. A robust NLP model must capture all possible perspectives of a sequence before generating an output.

Sub-Layer 2: Feed-Forward Neural Network (FFN) Topology

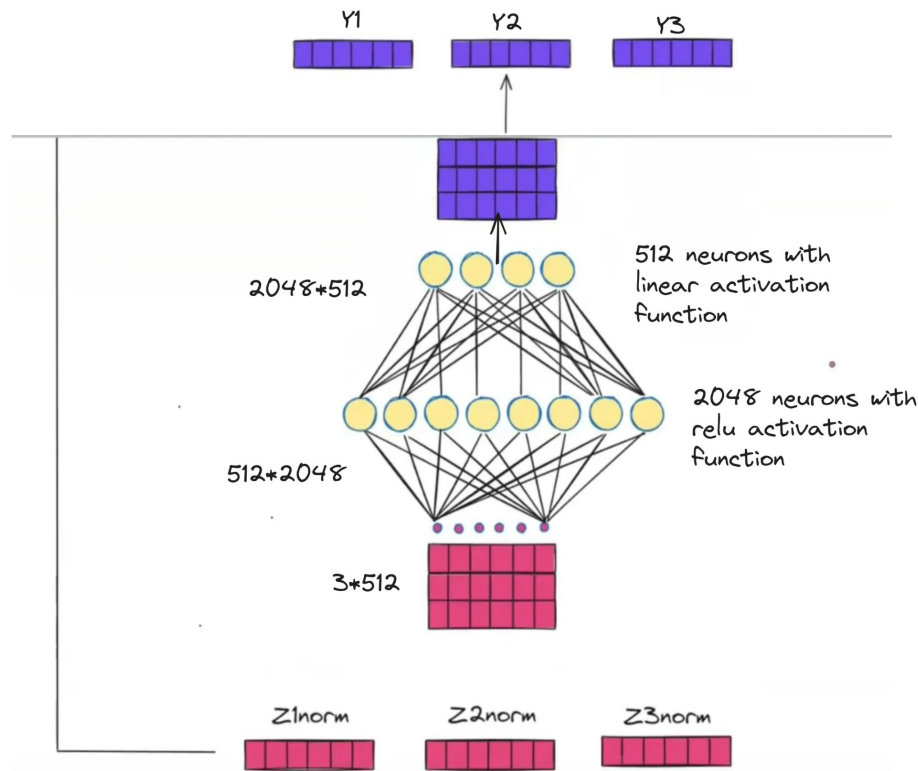


$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

A two-layer Multi-Layer Perceptron (MLP) applied identically but independently to every sequence position.

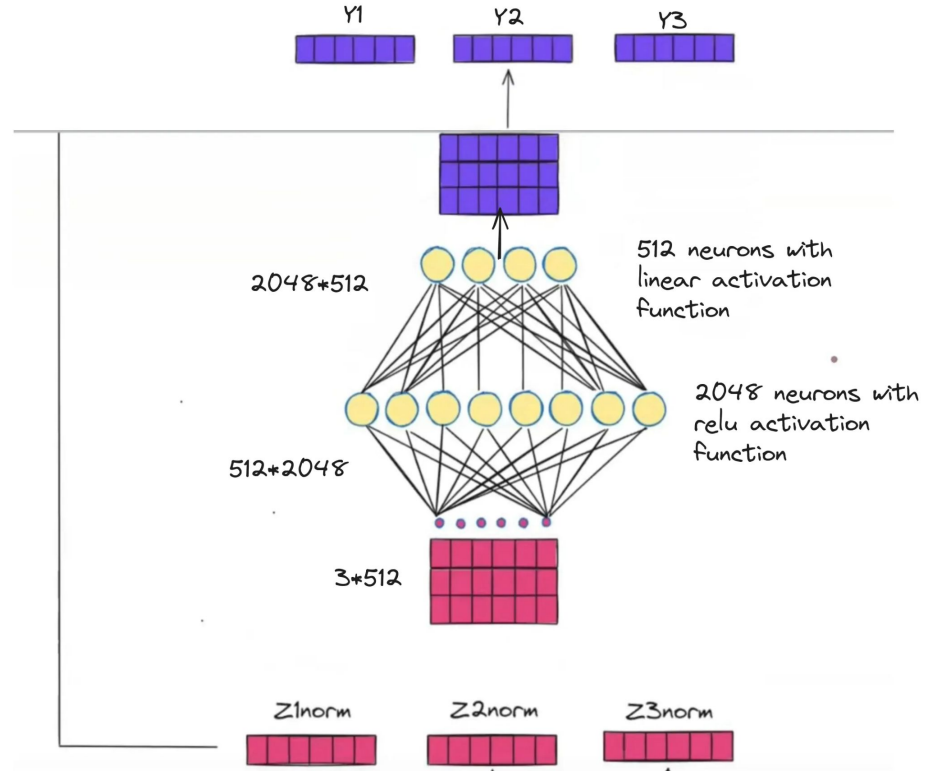
Feed forward network

- After layer normalization, the normalized vectors, $Z1_norm$, $Z2_norm$, and $Z3_norm$, are passed through a Feed Forward Neural Network (FFNN). Let's break down its architecture as described in the research paper:
 - The input layer is not counted as part of the neural network, but it receives the 512-dimensional input vectors.
 - The feed-forward network consists of two layers:
 - First layer with 2048 neurons and a ReLU activation function.
 - Second layer with 512 neurons and a linear activation function.



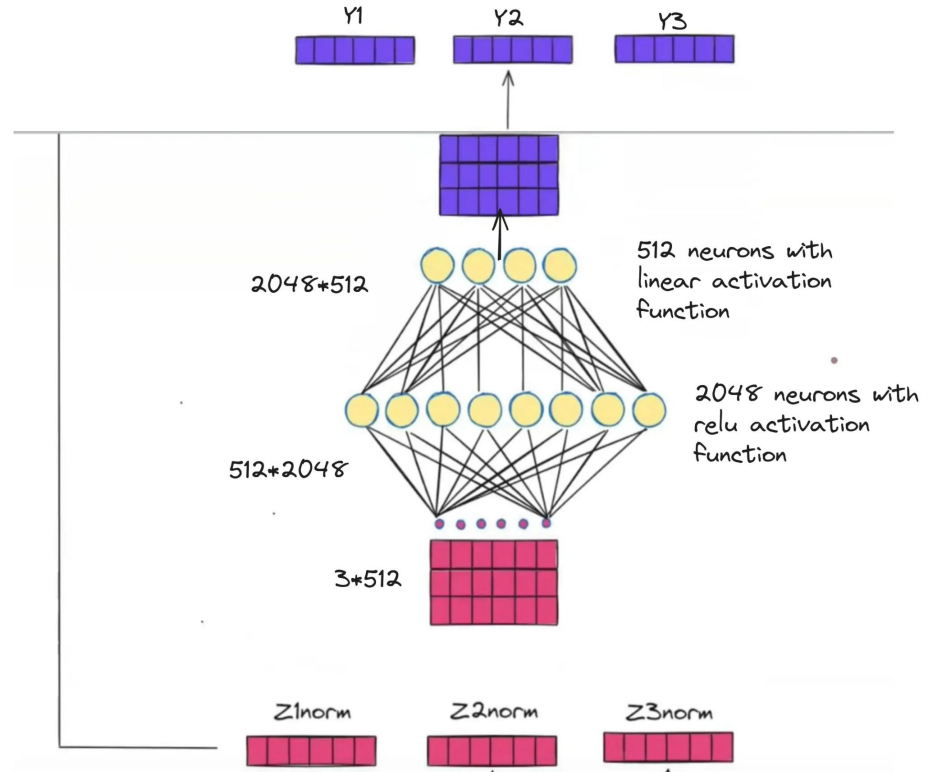
Feed forward network : Weights and Biases

- The weights between the input and the first layer form a 512×2048 matrix, represented as $W1$.
- Each of the 2048 neurons in the first layer has its own bias, represented collectively as $B1$.
- The weights between the first and second layer form a 2048×512 matrix, represented as $W2$.
- Each of the 512 neurons in the second layer has its own bias, represented collectively as $B2$.



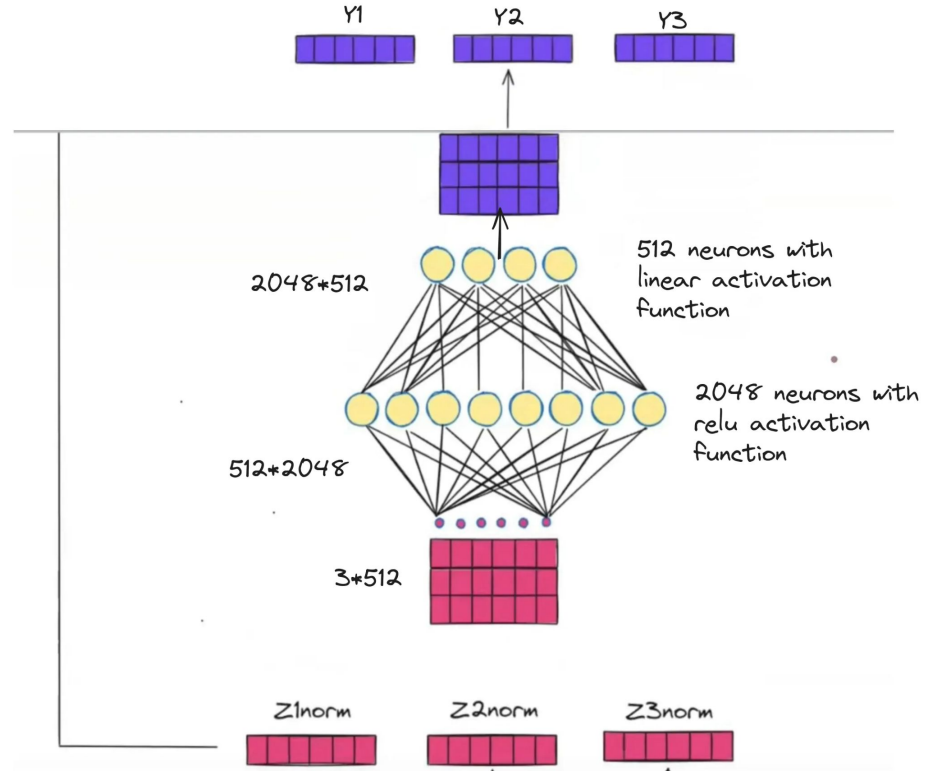
Feed forward network : Processing the input

- The input vectors $Z1_norm$, $Z2_norm$, and $Z3_norm$ can be imagined as stacked together to form a 3×512 matrix, where each row corresponds to one vector. This matrix is then fed into the FFNN.
- The input matrix is multiplied by the weights $W1$ and the bias $B1$ is added.
- A ReLU activation is applied to introduce non-linearity.
- The output is a 3×2048 matrix, representing the expanded dimensionality.
- This matrix is multiplied by the weights $W2$ and bias $B2$ is added, resulting in a 3×512 matrix.
- Essentially, the dimensionality of the input vectors is first increased from 512 to 2048, and then reduced back to 512.

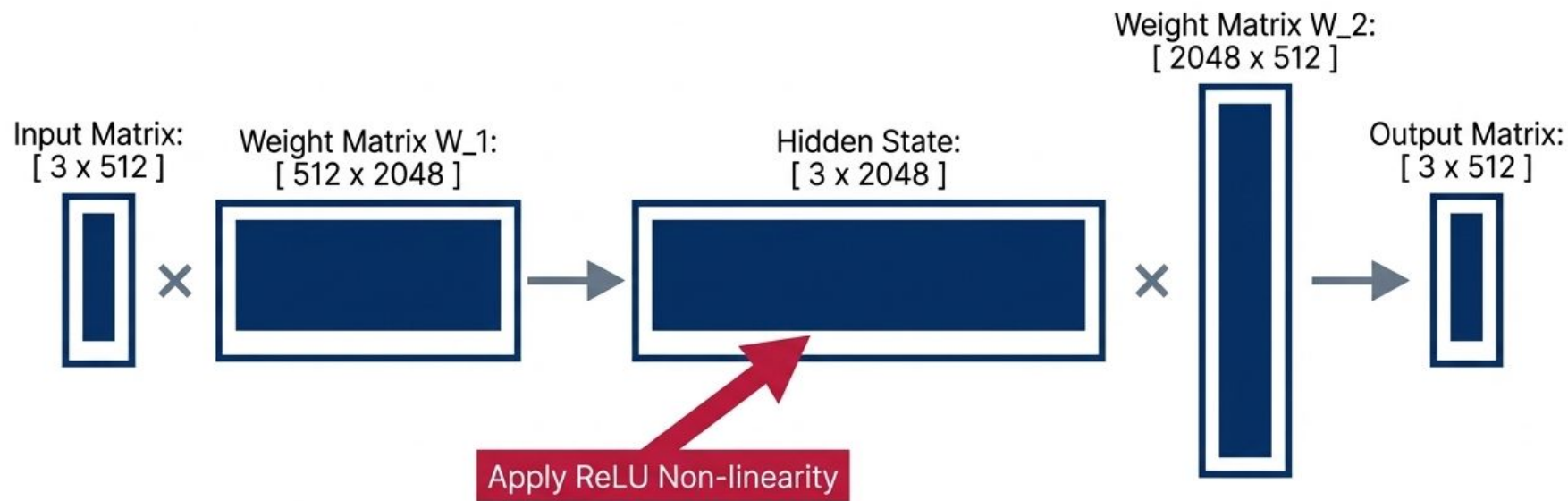


Feed forward network : Final Output

- The final result is a set of three vectors, each with 512 dimensions, similar to the input. Let's call these vectors Y1, Y2, and Y3.



Matrix Dimensionality Operations within the FFN



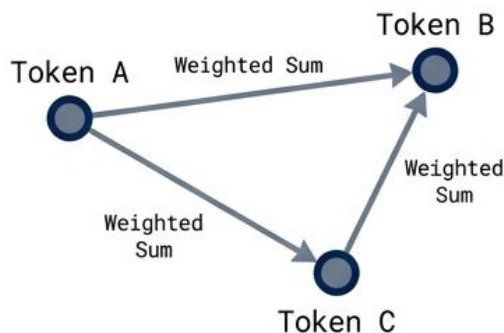
Mechanism: The network intentionally projects the 512-dimensional representation into a massive 2048-dimensional space.

Rationale: Expanding dimensionality allows the network to tease apart complex features and apply robust non-linear transformations before compressing the data exactly back to its 512-d base state.

The Theoretical Role of the FFN: Why use it?

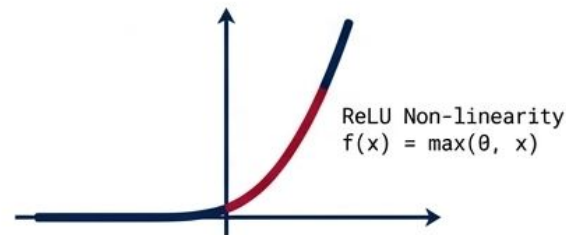
Multi-Head Attention

- **Operation:** Fundamentally Linear (weighted sums).
- **Role:** Context Gathering. Routing information between different tokens in the sequence.



Feed-Forward Network

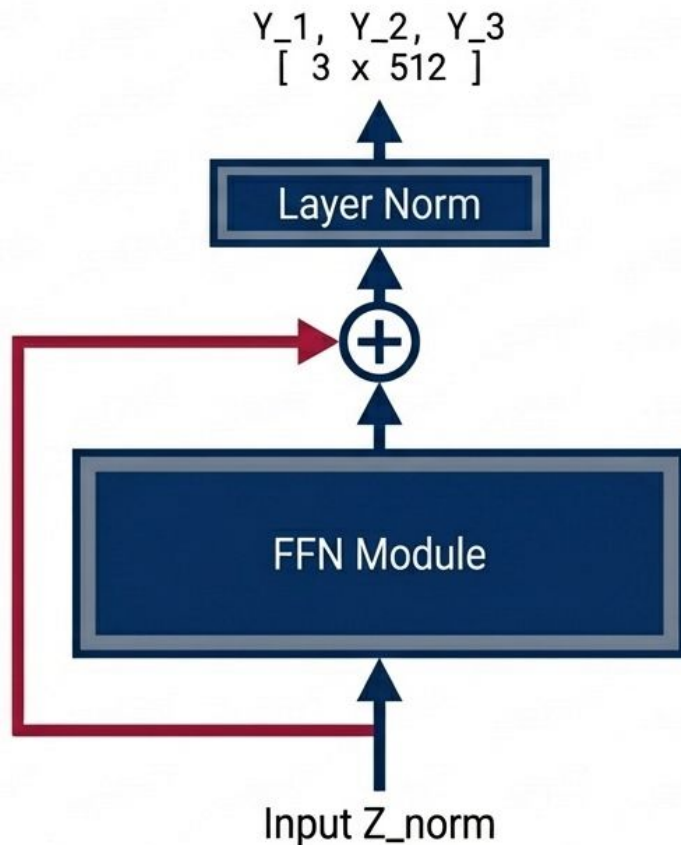
- **Operation:** Non-Linear (via ReLU activation).
- **Role:** Feature Extraction & Memory.



Transformer Feed-Forward Layers Are Key-Value Memories.

Emerging research suggests the massive 2048-d hidden layers act as parameterized memory, recalling learned factual associations to enrich the contextualized vectors.

Sub-Layer 2 Post-Processing: The Second Add & Norm

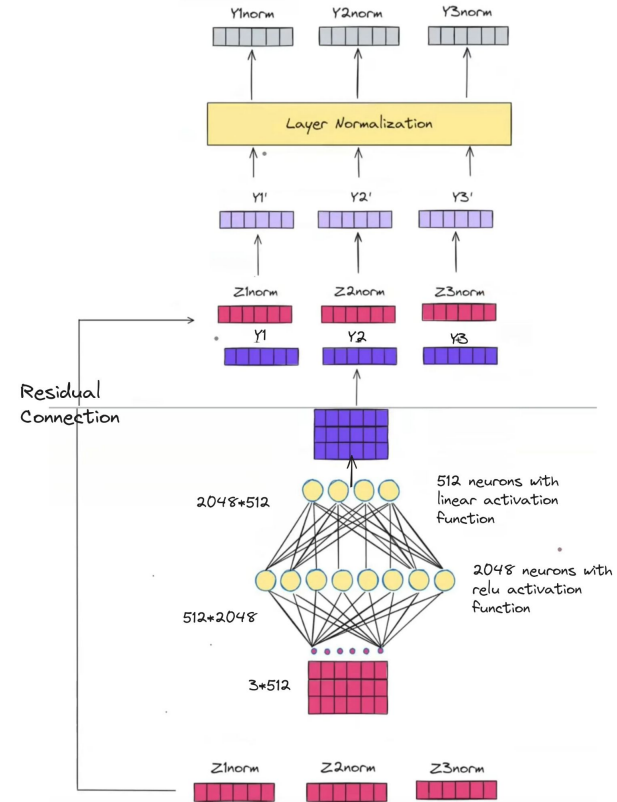


The resulting Y tensor represents data that has been fully contextualized (MHA), non-linearly transformed (FFN), and mathematically stabilized (Norm).

The block's structural circuit is now complete.

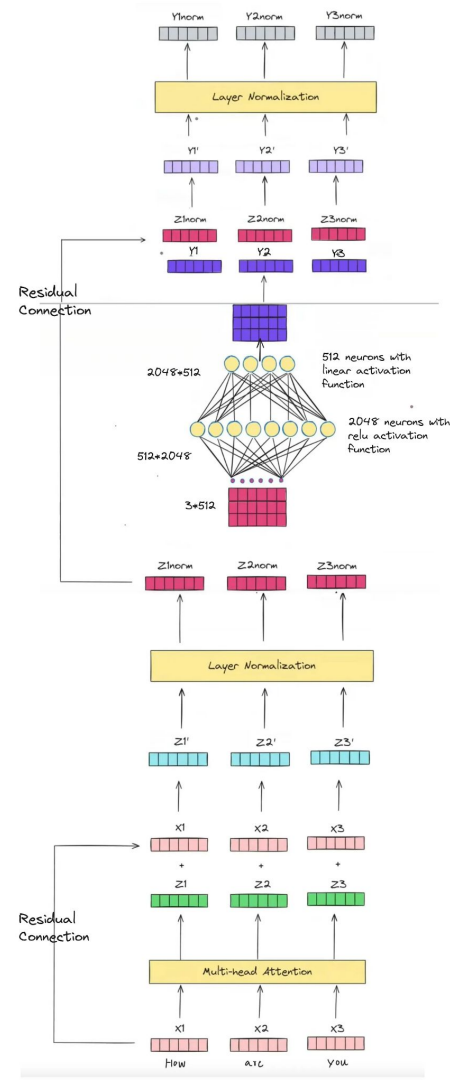
Feed forward network : Addition and Normalization for skip connections

- After the feed-forward network, we obtain three vectors, Y_1 , Y_2 , and Y_3 , each with a dimensionality of 512.
- Now, we perform an add operation. The original input vectors $Z1_norm$, $Z2_norm$, and $Z3_norm$ are bypassed and added to the output vectors Y_1 , Y_2 , and Y_3 , respectively. This results in a new set of vectors, which we'll call Y_1' , Y_2' , and Y_3' . All these vectors are still 512-dimensional.
- After this we apply LayerNorm again



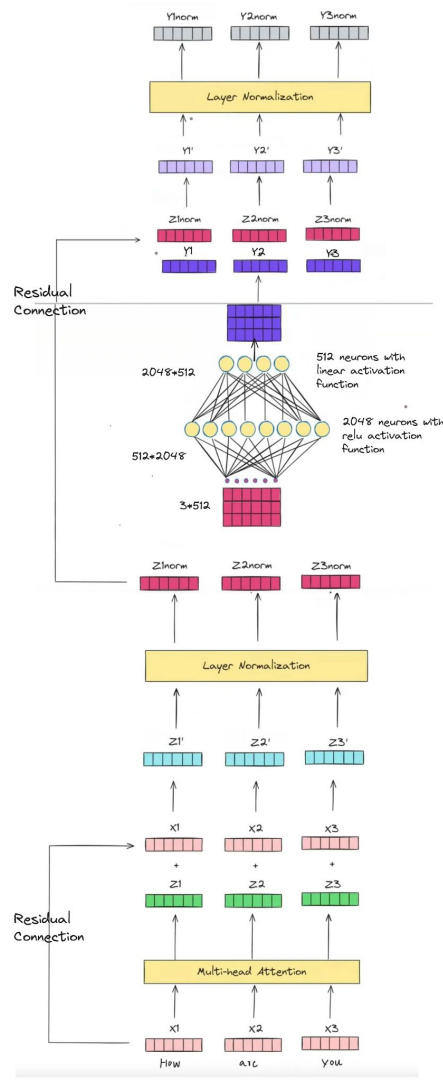
Next encoder block

- These normalized vectors $Y1_norm$, $Y2_norm$, and $Y3_norm$ are then passed as inputs to the next encoder block.
- In the next encoder block, the same operations will occur:
 - Multi-head attention will be applied.
 - Add & normalize will follow.
 - The output will then be processed through another feed-forward network.
 - Again, we'll have an add & normalize step before passing the vectors to the next encoder block.



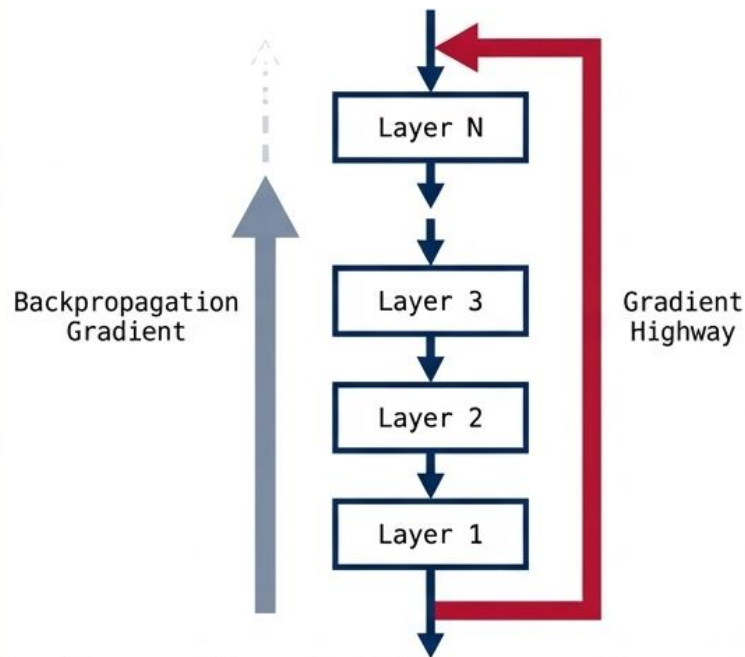
This process is repeated across a total of six encoder blocks, after which the output is passed to the decoder portion of the transformer.

The output of the very last Encoder layer is a sequence of vectors that contain the "meaning" of the entire input sentence.



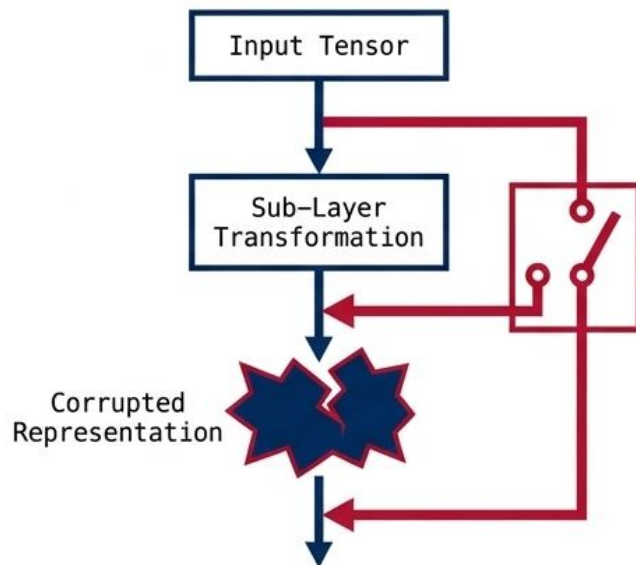
Architectural Deep Dive: The Rationale for Residual Connections

1. Stable Training (Mitigating Vanishing Gradients)



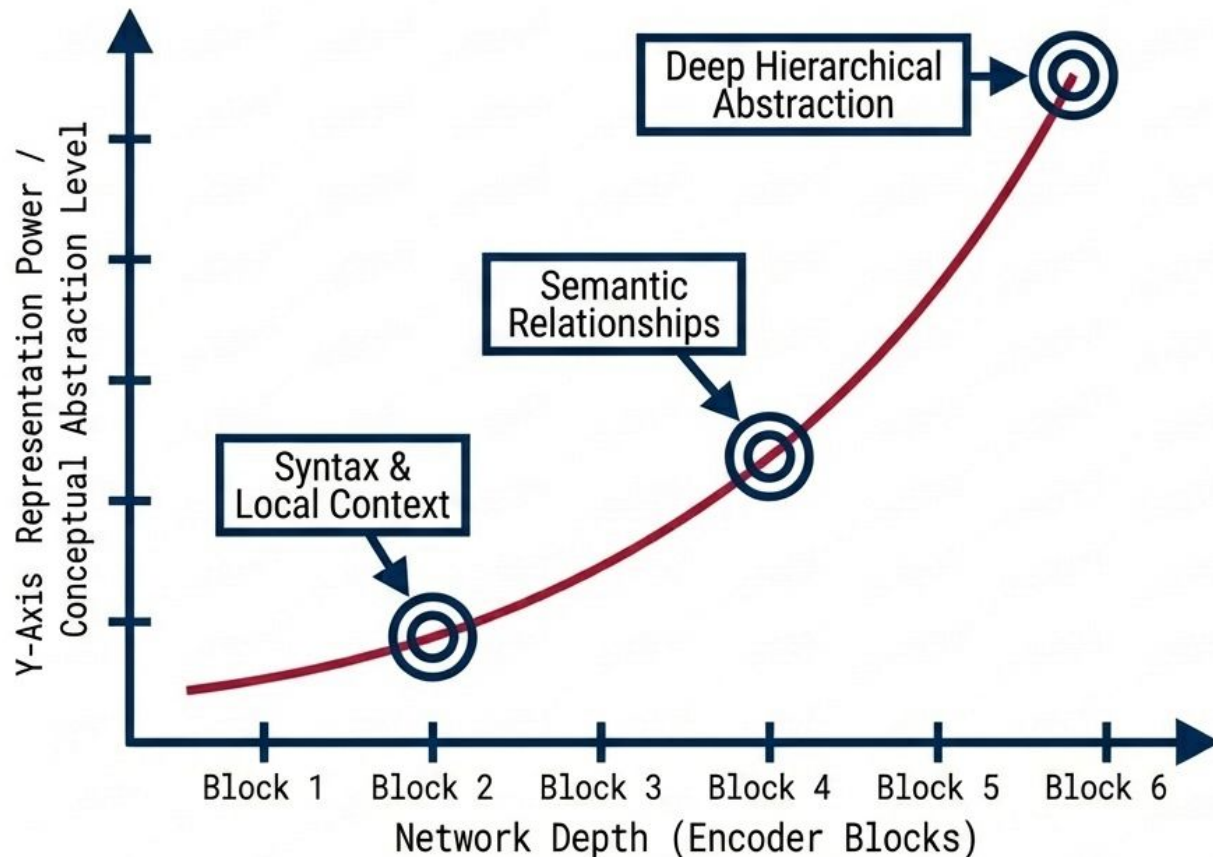
Provides an alternate highway, preventing gradients from vanishing into zero as they propagate backward through deep networks.

2. Preserving Feature Identity



If a sub-layer learns a detrimental transformation, the network can functionally skip it, relying on the uncorrupted identity mapping of the input features.

System Depth: Why Stack N=6 Blocks?

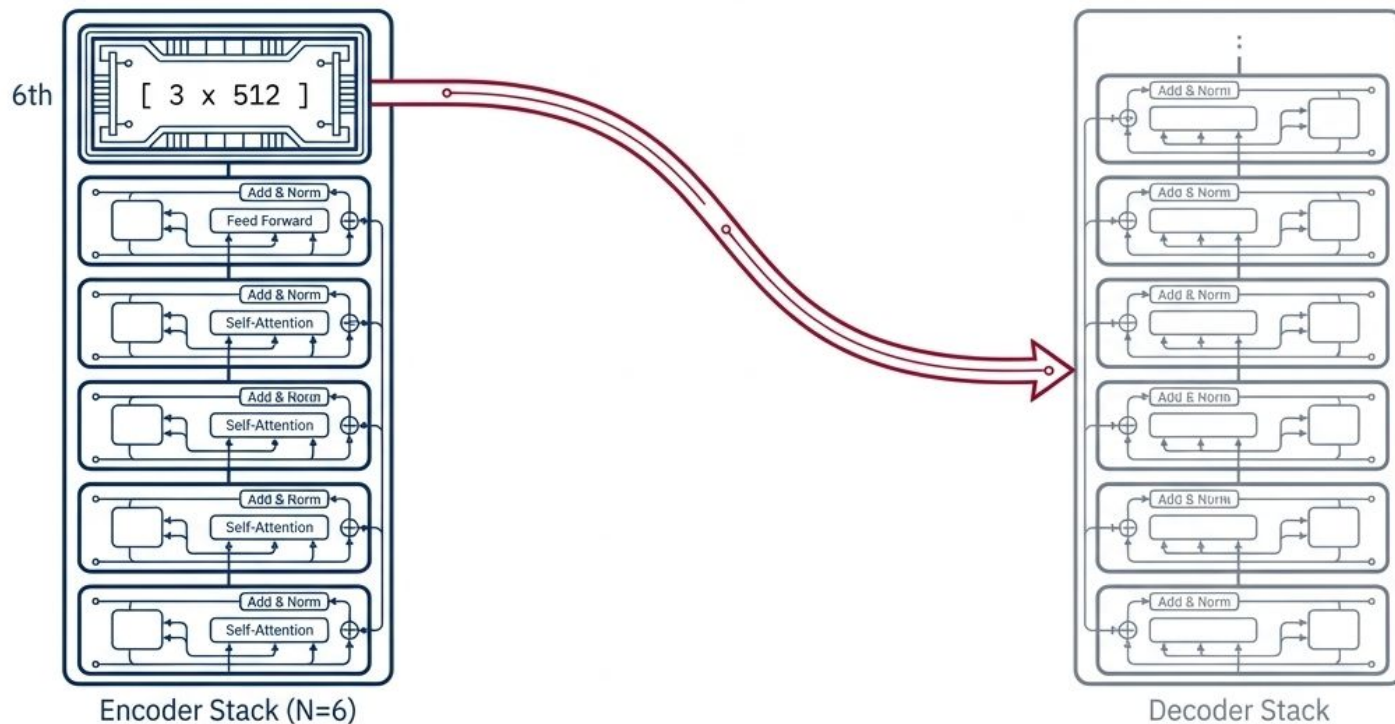


Human language is incredibly nuanced, deeply nested, and highly abstract.

A single encoder block lacks the representation power to decode this complexity.

Stacking multiple layers forces the network to refine the tensor sequentially, learning increasingly higher-order mathematical representations of the input sequence.

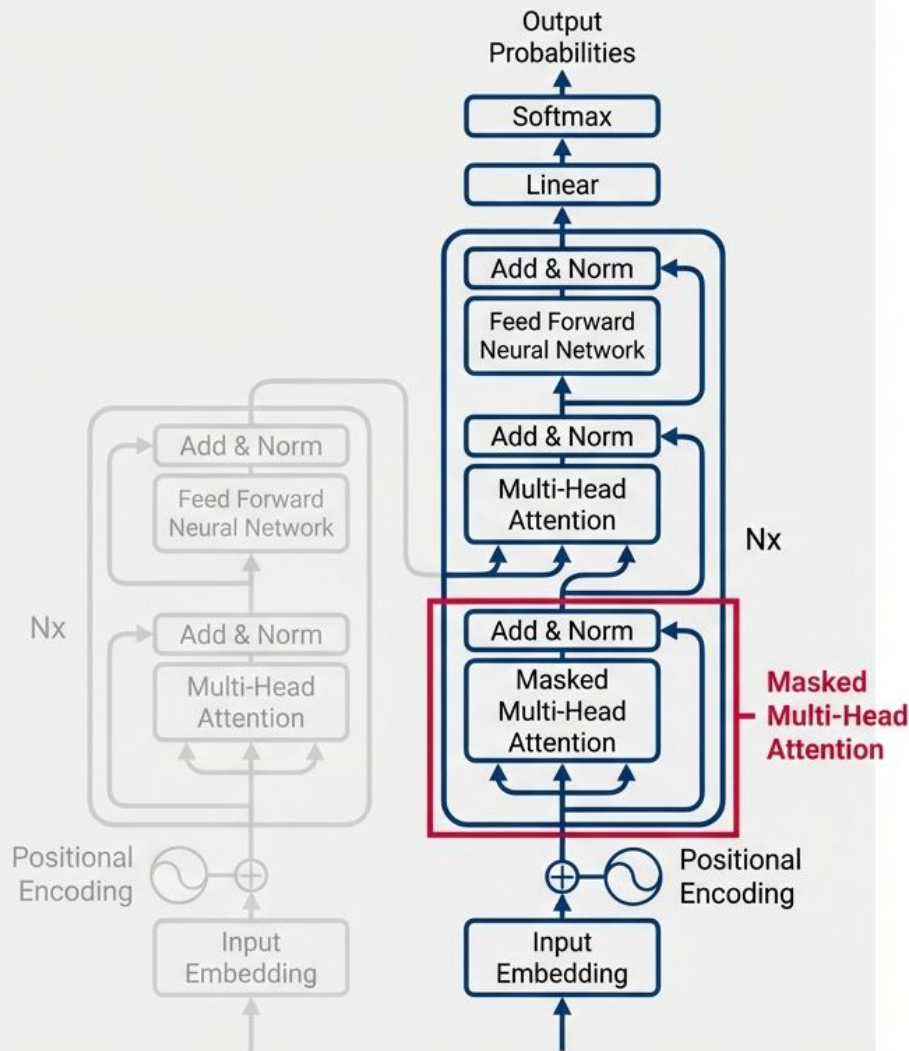
Bridging the Architecture: From Encoder to Decoder



The Culmination of the Encoder Phase:

- After passing through $N=6$ identical blocks, the final output matrix retains the exact $[\text{Seq_Length} \times 512]$ base shape it started with.
- This perfectly refined, highly abstract, context-rich tensor is now ready to be consumed by the Decoder stack to generate final predictions.

Decoder Architecture



Transitioning to the Decoder: The 50% Milestone

Encoder Mastery Complete

We have established the Encoder's ability to process complete input sequences simultaneously via Self-Attention, capturing bidirectional context without time-step constraints.

The Decoder's Unique Constraint

Unlike the Encoder, the Decoder operates under strict directional rules. It shares structural DNA with the Encoder (Add & Norm, Feed Forward layers), but introduces a critical mathematical modification at its foundation to prevent seeing the future.

The Fundamental Theorem of Decoder Execution

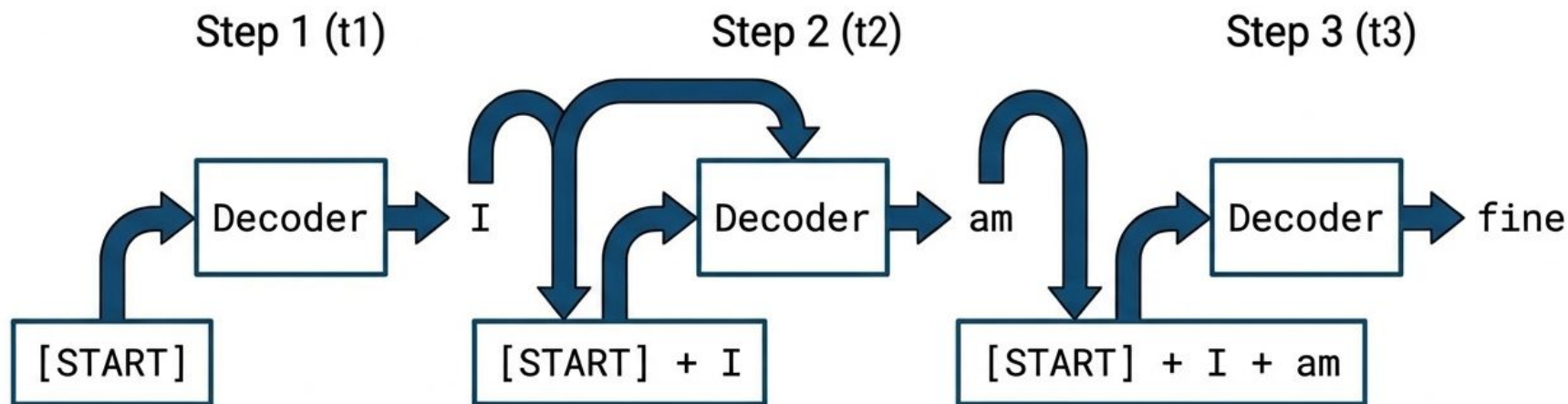
“The Transformer Decoder behaves as an Auto-Regressive model at inference time, but operates as a Non-Auto-Regressive model at training time.”

Inference	Training
Auto-Regressive	Non-Auto-Regressive
Sequential, step-by-step generation. Dependent on past outputs.	Parallel, simultaneous processing. Independent of past outputs.

The Architectural Paradox: How can a single, static mathematical architecture support two entirely different execution paradigms?

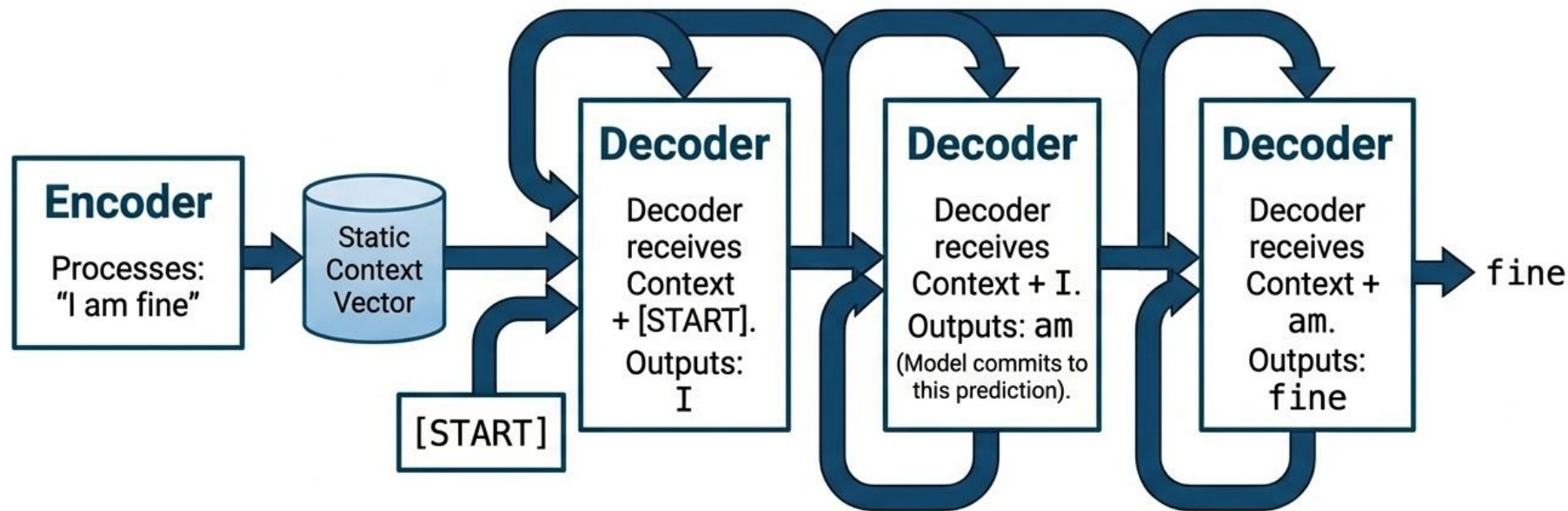
The Auto-Regressive Constraint (Time-Series Inheritance)

In Deep Learning, auto-regressive models generate data points in a sequence by conditioning each new point *strictly* on previously generated points.



Sequential data is inherently dependent. The model cannot structurally generate 'fine' without first mathematically establishing 'I'.

Inference: The Mandatory Sequential Loop



$O(N)$ Time Complexity. During inference, the model does not know the future. To generate a 300-word paragraph, the Decoder must execute its entire multi-layer internal stack 300 separate times.

Introduction to Training vs. Inference in Transformers

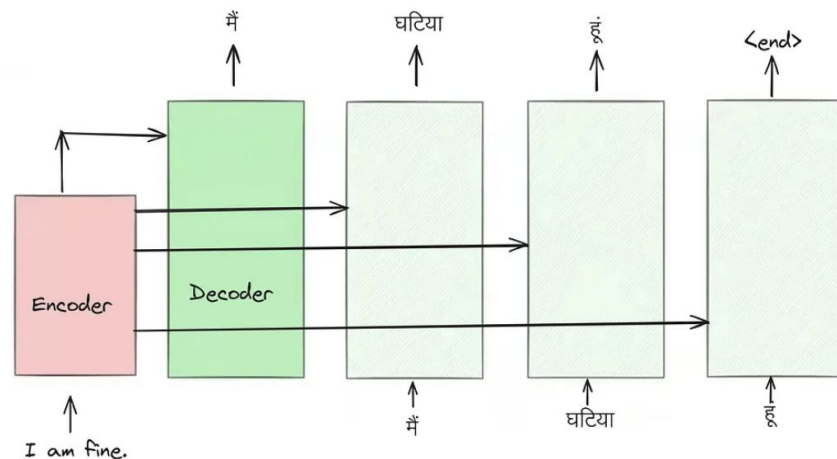
The Core Difference: The Transformer decoder is *autoregressive* at inference time but *non-autoregressive* at training time.

Autoregressive (Inference): The model generates each token sequentially, conditioning each new token on the previously generated ones (e.g., word 2 depends on word 1).

The Inference Challenge: If the model makes an early mistake (e.g., predicting "bad" instead of "fine"), the error cascades because the incorrect word is fed into the next step.

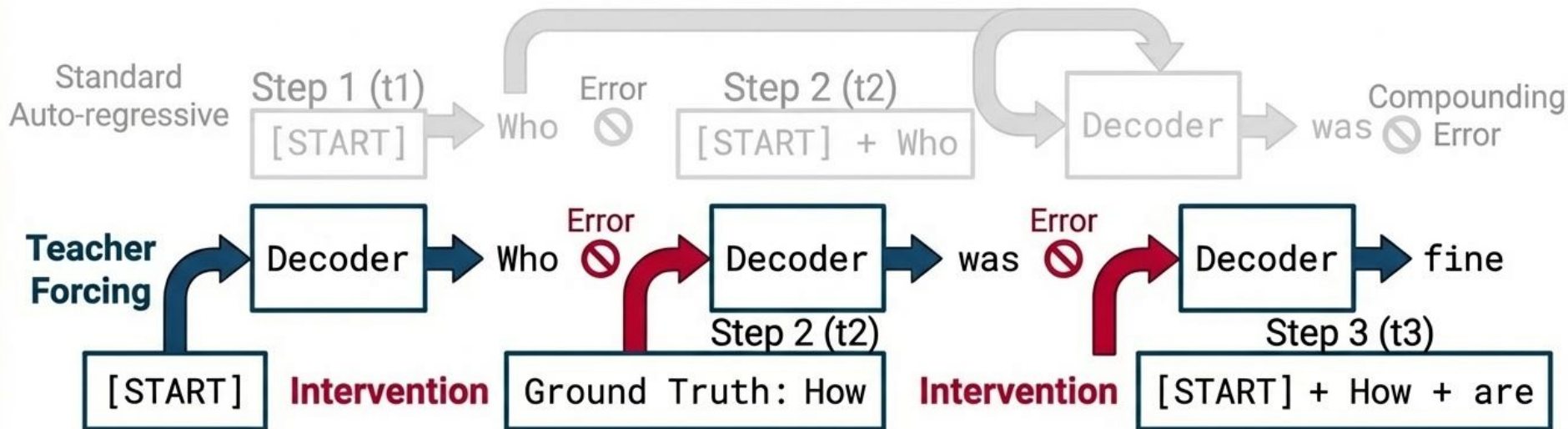
Inference

Query Sentence -> I am fine



Training: Breaking the Chain via Teacher Forcing

During training, we possess the absolute ground-truth target sequence. We do not need to wait for the model's predictions to feed the next time-step.



Key Insight: Because inputs are predefined by the dataset, the dependency on the previous time-step's output is eliminated.

The Shift from Sequential to Parallel

Execution Mode	Input Method	Time Complexity	Hardware Utilization
Auto-Regressive Training	Wait for $Y_{(t-1)}$ to input X_t	$O(N)$ per sequence	Poor GPU utilization
Non-Auto-Regressive	Feed all ground truth tokens simultaneously	$O(1)$ per sequence	Maximizes GPU parallelization

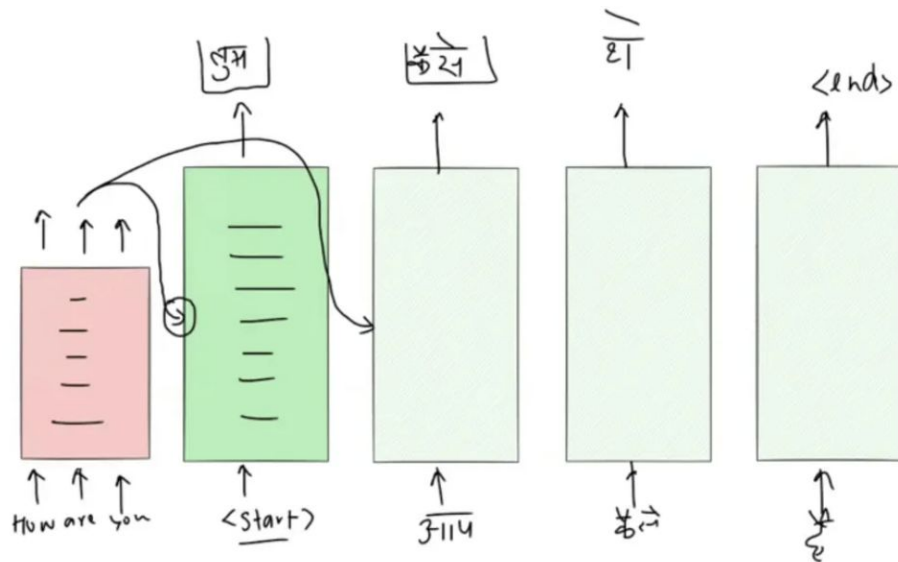
The Optimization Hypothesis: If all inputs ([START], How, are) are known immediately via Teacher Forcing, the Decoder's operations can execute in massive parallel matrices rather than slow sequential loops.

The Training Phase & Teacher Forcing

Sequential Training Problem: Treating training like inference (word-by-word) is too slow for large datasets. A 300-word paragraph would require 300 individual sequential passes.

Teacher Forcing: During training, we already have the correct target dataset. Instead of feeding the model's wrong prediction into the next step, we feed it the *actual correct word*.

Parallelization: Because we know all the correct output words beforehand, we don't have to wait for the model to predict them one by one. We can pass the entire correct sequence into the decoder simultaneously.

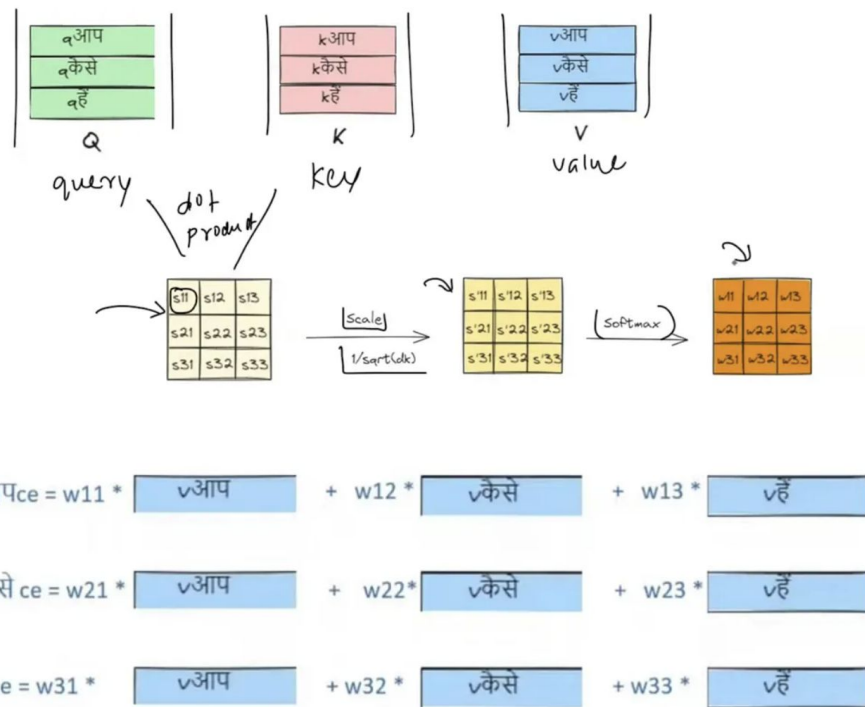


The Danger of Data Leakage

The Parallel Processing Trap: When feeding all words into the self-attention block simultaneously for speed, a severe problem arises: the model can "see the future."

Data Leakage: Self-attention uses information from all tokens. If calculating the embedding for token 1, it unfairly uses context from token 2 and 3—which wouldn't exist yet during real-world inference.

The Result: The model "cheats" and performs perfectly in training, but fails to generalize during step-by-step inference because it relied on future context it no longer has.



The Threat of Data Leakage (Cheating)

If we feed all target tokens into a standard Self-Attention block simultaneously to achieve parallel speed, we trigger a catastrophic flaw. The model sees the future.



Mathematical Representation of Contextual Embeddings

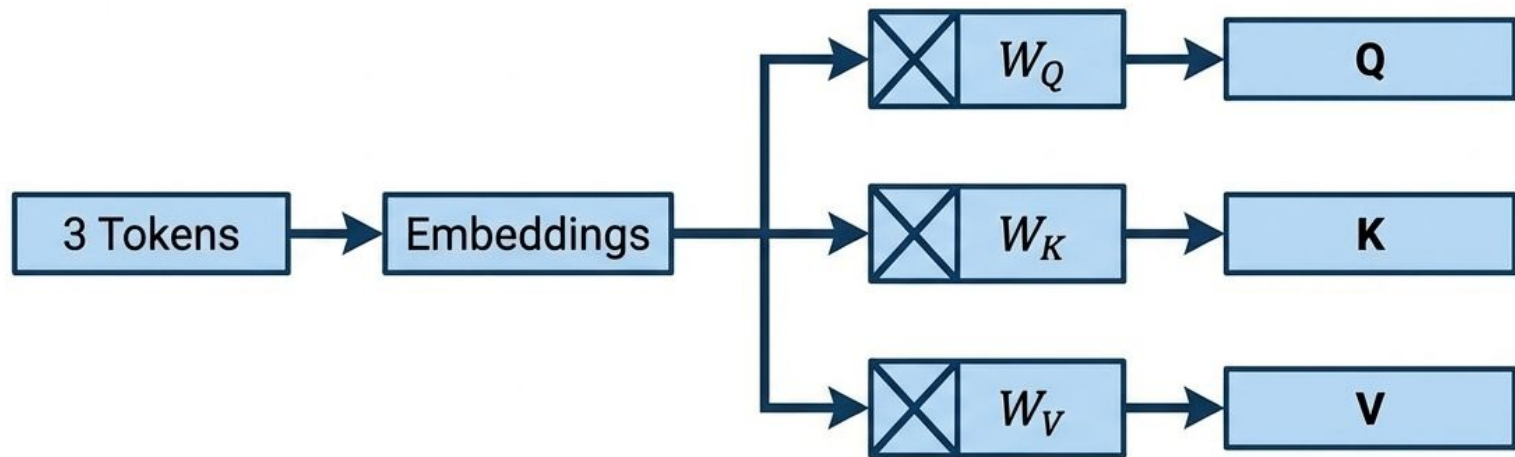
$\text{Context}(\text{How}) = 0.8(\text{How}) + 0.1(\text{are}) + 0.1(\text{you}) \rightarrow \text{ILLEGAL}$

$\text{Context}(\text{are}) = 0.1(\text{How}) + 0.8(\text{are}) + 0.1(\text{you}) \rightarrow \text{ILLEGAL}$

$\text{Context}(\text{you}) = 0.1(\text{How}) + 0.2(\text{are}) + 0.7(\text{you}) \rightarrow \text{LEGAL}$

The token 'How' is deriving its mathematical context from 'are' and 'you'—words that have not been 'generated' yet in sequential time.

Finding the Intervention Point: Self-Attention Recap



Q (Query):
What I am looking for.

K (Key):
What I contain.

V (Value):
What I contribute.

Concept Check: Up to this exact point, computations are strictly parallel and isolated. **Q**, **K**, and **V** vectors are generated independently. No leakage has occurred yet.

The Genesis of Leakage: $Q \cdot K^T$

Leakage occurs during the computation of raw Attention Scores. Every Query is dotted against every Key to determine relevance. The upper triangular portion inherently represents connections to future tokens.

	Key (How)	Key (are)	Key (you)
Query (How)	Valid	Leakage	Leakage
Query (are)	Valid	Valid	Leakage
Query (you)	Valid	Valid	Valid